

MODULE 38

SQL and Query Performance with PostgreSQL

Master joins, subqueries, and indexing, then read real EXPLAIN ANALYZE plans to find and fix slow PostgreSQL queries.





MODULE 38 OVERVIEW

Write efficient SQL and use PostgreSQL's execution plans to keep queries fast at scale.

Northstar CRM's customer and account screens must stay fast as data grows -- every query in this module is tuned against the exact schema Module 37 built.

DURATION & LAB



1.5 Hours Theory

SQL review, joins, subqueries, aggregation, indexing deep dive, and reading execution plans.



2 Hours Lab

Diagnose and fix real slow queries against the CRM schema using EXPLAIN ANALYZE.



Scenario

Northstar CRM customer and account search screens under real production-scale data.

MODULE ARC



Master SQL & Joins

Review SELECT/WHERE and combine tables with INNER, LEFT, subqueries, and CTEs.



Go Deep on Indexes

Selectivity, composite column order, partial and expression indexes.



Read & Tune Plans

EXPLAIN ANALYZE, scan and join types, and a repeatable tuning workflow.

KEY TAKEAWAY Total time: 3.5 hours (1.5 hours theory + 2 hours hands-on lab). By the end, you will read a PostgreSQL execution plan and fix the query it describes.

WHY SQL PERFORMANCE MATTERS

Every application depends on the database -- an unoptimized query can be fast today and crippling tomorrow.



Slow Response Times

Users experience delays and timeouts, leading to frustration and lost productivity.



Higher Infrastructure Costs

Inefficient queries burn more CPU, memory, and I/O, requiring more hardware.



Poor Scalability

A query that works at 1,000 rows can fall over at 1,000,000 without ever changing its SQL.



Business Impact

Slow reports, delayed decisions, and missed SLAs affect business outcomes directly.

KEY TAKEAWAY Slow queries don't just feel sluggish -- they chain directly into higher infrastructure costs, worse scalability, and real business impact.



Why SQL Performance Matters

Efficient SQL. Faster Applications. Happier Users. Lower Costs.



In enterprise systems, SQL queries run millions—sometimes billions—of times every day. Even small inefficiencies can lead to significant slowdowns, high resource usage, and increased costs. Optimizing SQL is essential for delivering scalable, reliable, and cost-effective data solutions.



BUSINESS IMPACT



Slower Application Response

Inefficient queries increase page load times and degrade user experience.



Higher Infrastructure Costs

Poorly written SQL causes unnecessary CPU, memory, and I/O usage.



Scalability Challenges

As data grows, slow queries become bottlenecks that limit system scalability.



Business Impact

Delays in reports, analytics, and transactions can affect critical business decisions.

THE RIPPLE EFFECT OF SLOW SQL



KEY TAKEAWAY

Great applications start with great SQL. Optimized queries drive performance, efficiency, and business value.

WHAT GOOD SQL DELIVERS

- ✓ Faster query execution and response times
- ✓ Efficient resource utilization
- ✓ Better scalability and system stability
- ✓ Lower infrastructure and maintenance costs
- ✓ Improved user satisfaction
- ✓ Reliable reporting and analytics



Writing efficient SQL is not just a **technical skill**—it's a **business advantage**.

In this module, you will learn how to analyze, optimize, and tune SQL for maximum performance.

MODULE LEARNING OBJECTIVES

By the end of this module, you will write efficient SQL and read a PostgreSQL execution plan to diagnose slow queries.

1. Write Correct SQL -- review SELECT, WHERE, and combine tables with every join type.
2. Query Flexibly -- use subqueries, correlated subqueries, and CTEs where each fits best.
3. Summarize Data -- apply GROUP BY, HAVING, and window functions correctly.
4. Index Deliberately -- reason about selectivity, column order, and specialized index forms.
5. Understand the Planner -- explain how PostgreSQL's cost-based optimizer chooses a plan.
6. Read Execution Plans -- interpret EXPLAIN and EXPLAIN ANALYZE output confidently.
7. Recognize Plan Nodes -- name common scan and join types on sight.
8. Tune Systematically -- apply a repeatable measure-diagnose-fix-verify workflow.

KEY TAKEAWAY Goal: read a real PostgreSQL execution plan and know exactly what to change to make a slow query fast.



LEARNING OBJECTIVES

Module 38: SQL and Query Performance

By the end of this module, you will be able to understand, analyze, and optimize SQL queries in PostgreSQL to improve application performance and support scalable, efficient systems.



FOCUS

Build the skills to write efficient SQL, understand how PostgreSQL executes queries, and apply proven tuning techniques for maximum performance.



Outcome: Write better SQL.
Diagnose performance issues.
Deliver faster, more scalable solutions.

IN THIS MODULE, YOU WILL:



Review core SQL concepts and advanced queries



Work with joins, subqueries, CTEs, aggregations, and window functions



Understand how PostgreSQL uses indexes to optimize queries



Explore the query planner and cost-based optimizer



Analyze execution plans with **EXPLAIN** and **EXPLAIN ANALYZE**



Identify performance bottlenecks and inefficient patterns



Apply indexing strategies and follow best practices



Optimize queries using proven tuning techniques



Maintain performance with statistics, **ANALYZE**, and **VACUUM**



Measure performance improvements and validate results



These objectives will empower you to deliver high-performance SQL solutions that drive **better user experiences** and **business outcomes**.

SQL QUERY EXECUTION OVERVIEW

Every SQL statement passes through the same four stages before a result set comes back.

1

Parse

Check syntax and validate that referenced tables and columns exist.

2

Plan

The optimizer chooses the cheapest execution strategy it can find.

3

Execute

PostgreSQL carries out the chosen plan -- scans, joins, sorts, filters.

4

Return

The final result set streams back to the client.

KEY TAKEAWAY Every query is Parsed, Planned, Executed, and Returned -- EXPLAIN, covered later this module, shows exactly what happens in the Plan and Execute stages.



SQL Query Execution Overview

From SQL Statement to Result Set

When you run an SQL query, PostgreSQL follows a well-defined process to return the result set.

The database parses your request, creates an execution plan, retrieves the data efficiently, and returns the results.

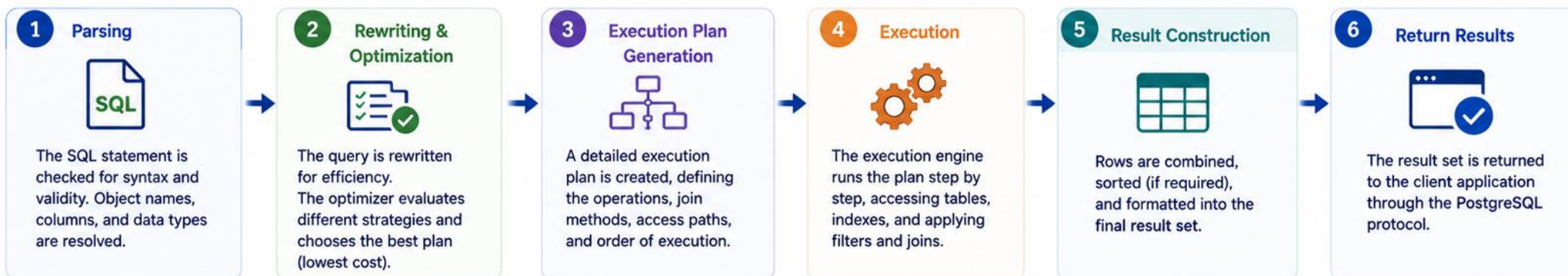
SQL Statement

```
SELECT c.customer_id, c.name, o.order_date, o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2024-01-01'
AND o.status = 'ACTIVE';
```

Result Set

customer_id	name	order_date	total_amount
101	Alice	2024-01-05	1250.00
105	Bob	2024-01-06	980.00
107	Carol	2024-01-07	1580.00
...	...	...	...

THE SQL QUERY EXECUTION PIPELINE



KEY INSIGHTS

- The optimizer's goal is to minimize the total cost (I/O, CPU, memory).
- Indexes, statistics, and query structure greatly influence the plan chosen.
- Understanding execution plans helps identify bottlenecks and tuning opportunities.

TOOLS TO EXPLORE

- **EXPLAIN** – shows the chosen execution plan.
- **EXPLAIN ANALYZE** – shows the actual execution with runtime and row counts.
- **pg_stat_statements** – helps find slow queries in production.



Understanding how SQL queries are executed is the **foundation for writing efficient SQL and building scalable, high-performance data applications.**



SELECT REVIEW

SELECT retrieves data from one or more tables -- the most frequently used SQL statement, worth reviewing precisely.

```
SELECT customer_id, full_name, status
FROM customer
WHERE status = 'ACTIVE'
ORDER BY full_name
LIMIT 10;
```

SELECT names the columns to return -- prefer explicit columns over * for performance and clarity.

FROM names the source table; WHERE filters rows; ORDER BY sorts; LIMIT caps the row count.

Best practice: select only the columns the caller actually needs.

KEY TAKEAWAY Always select only the columns you need instead of SELECT * -- it reduces network traffic and sets up the sargability and index-only-scan discussions later in the module.



SELECT Review

The Foundation of SQL Queries

The SELECT statement is used to retrieve data from one or more tables. It is the most commonly used SQL command and the building block for all other queries.

Basic SELECT Syntax

```
SELECT column1, column2, ...  
FROM table_name  
[WHERE condition]  
[ORDER BY column [ASC|DESC]];
```

What SELECT Does

- ✓ Retrieves data from one or more tables or views
- ✓ Allows filtering, sorting, and limiting of data
- ✓ Returns a result set (rows and columns)
- ✓ Read-only operation (safe to run)

COMMON SELECT CLAUSES

CLAUSE	PURPOSE	EXAMPLE	SAMPLE RESULT												
 SELECT	Specifies the columns to retrieve.	<pre>SELECT id, name, email FROM customers;</pre>	<table><tr><th>id</th><th>name</th><th>email</th></tr><tr><td>1</td><td>Alice</td><td>alice@example.com</td></tr><tr><td>2</td><td>Bob</td><td>bob@example.com</td></tr></table>	id	name	email	1	Alice	alice@example.com	2	Bob	bob@example.com			
id	name	email													
1	Alice	alice@example.com													
2	Bob	bob@example.com													
 WHERE	Filters rows based on a condition.	<pre>SELECT * FROM customers WHERE status = 'ACTIVE';</pre>	<table><tr><th>id</th><th>name</th><th>status</th></tr><tr><td>1</td><td>Alice</td><td>ACTIVE</td></tr><tr><td>3</td><td>Carol</td><td>ACTIVE</td></tr></table>	id	name	status	1	Alice	ACTIVE	3	Carol	ACTIVE			
id	name	status													
1	Alice	ACTIVE													
3	Carol	ACTIVE													
 ORDER BY	Sorts the result set by one or more columns.	<pre>SELECT * FROM customers ORDER BY name ASC;</pre>	<table><tr><th>id</th><th>name</th><th>email</th></tr><tr><td>1</td><td>Alice</td><td>alice@example.com</td></tr><tr><td>2</td><td>Bob</td><td>bob@example.com</td></tr></table>	id	name	email	1	Alice	alice@example.com	2	Bob	bob@example.com			
id	name	email													
1	Alice	alice@example.com													
2	Bob	bob@example.com													
 LIMIT	Limits the number of rows returned.	<pre>SELECT * FROM customers ORDER BY id LIMIT 10;</pre>	<table><tr><th>id</th><th>name</th><th>email</th></tr><tr><td>1</td><td>Alice</td><td>alice@example.com</td></tr><tr><td>2</td><td>Bob</td><td>bob@example.com</td></tr><tr><td>...</td><td>...</td><td>...</td></tr></table>	id	name	email	1	Alice	alice@example.com	2	Bob	bob@example.com	...	...	...
id	name	email													
1	Alice	alice@example.com													
2	Bob	bob@example.com													
...	...	...													

BEST PRACTICES

-  Select only the columns you need (avoid SELECT *).
-  Use **WHERE** to filter early and reduce data volume.
-  Use **ORDER BY** for predictable result ordering.
-  Use **LIMIT** for pagination and testing large datasets.
-  Use meaningful column aliases for readability.



Mastering SELECT is the first step toward writing powerful, efficient, and reliable SQL queries.
Every advanced query builds on this foundation.

FILTERING WITH WHERE

WHERE filters rows based on a condition -- only rows where the condition evaluates to TRUE are returned.

Operator	Example
<code>=, <>, <, >, <=, >=</code>	<code>WHERE status = 'ACTIVE'</code>
BETWEEN	<code>WHERE created_at BETWEEN d1 AND d2</code>
IN	<code>WHERE status IN ('ACTIVE','PROSPECT')</code>
LIKE	<code>WHERE full_name LIKE 'Ami%'</code>
IS NULL / IS NOT NULL	<code>WHERE closed_at IS NULL</code>

KEY TAKEAWAY Filtering early with a sargable WHERE clause -- one an index can match directly -- is the single biggest lever over a query's performance.



Filtering with WHERE

The WHERE clause filters rows based on specified conditions.



KEY CONCEPTS

- WHERE filters rows before results are returned.
- Use comparison operators (=, <, >, <=, >=, <>, !=).
- Combine conditions using AND, OR, NOT.
- Use IN for multiple values.
- Use BETWEEN for ranges.
- Use LIKE for pattern matching.
- Use IS NULL / IS NOT NULL for NULL values.



WHY IT MATTERS

Filtering reduces the amount of data processed and returned, improving performance and ensuring accurate results.

EXAMPLE QUERIES

```
-- Simple filter
SELECT *
FROM employees
WHERE department = 'Engineering';

-- Multiple conditions
SELECT *
FROM employees
WHERE salary > 75000 AND experience >= 5;

-- Using IN and BETWEEN
SELECT *
FROM employees
WHERE department IN ('Engineering', 'IT', 'Analytics')
AND salary BETWEEN 60000 AND 100000;
```

SAMPLE DATA: employees

id	name	department	salary	experience
1	Alice Johnson	Engineering	95000	6
2	Bob Smith	IT	65000	3
3	Carol White	Engineering	120000	8
4	David Brown	Analytics	70000	4
5	Eva Green	HR	55000	2

Result of:

```
SELECT * FROM employees
WHERE department = 'Engineering'
AND salary > 75000;
```

id	name	department	salary
1	Alice Johnson	Engineering	95000
3	Carol White	Engineering	120000



BEST PRACTICE: Filter early and use indexed columns in WHERE conditions to improve query performance.



JOINING TABLES

A join combines rows from two or more tables based on a related column -- the core operation for querying a normalized schema.

Join Type	Returns
INNER JOIN	Only rows with a match in both tables.
LEFT JOIN	Every row from the left table, matched or not.
Multiple Joins	Three or more tables chained together in one query.
(RIGHT / FULL OUTER exist too)	Mirror of LEFT; union of matched and unmatched both sides.

KEY TAKEAWAY Every join names two tables and an ON condition -- the join type controls what happens to rows that don't have a match on the other side.

Joining Tables

Use JOINS to combine rows from two or more tables based on a related column.



KEY CONCEPTS

- Tables store related data in separate rows.
- JOIN combines rows from multiple tables based on a related column.
- The most common JOIN is INNER JOIN.
- Other JOIN types help include unmatched rows.
- JOIN improves query power and reduces the need for subqueries.

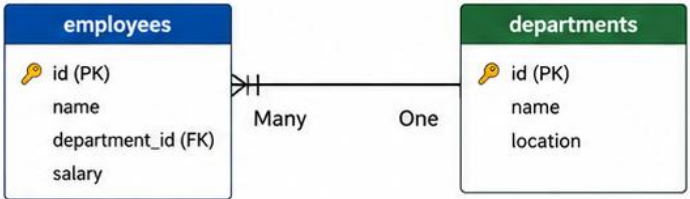


WHY IT MATTERS

- Brings related data together for meaningful results.
- Forms the foundation for reporting and analytics.
- Supports normalized database design.
- Improves performance by reducing data redundancy.



BEST PRACTICE: Always specify explicit JOIN conditions. Ensure JOIN columns are indexed for better performance.



Many employees belong to one department.

EXAMPLE: INNER JOIN

```
SELECT e.id, e.name, e.salary, d.name AS department, d.location
FROM   employees e
INNER JOIN departments d
      ON e.department_id = d.id
WHERE  e.salary > 60000
ORDER BY e.name;
```

employees			
id	name	department_id	salary
1	Alice Johnson	10	95000
2	Bob Smith	20	65000
3	Carol White	10	120000
4	David Brown	30	70000
5	Eva Green	20	55000

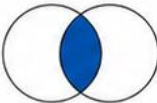
departments		
id	name	location
10	Engineering	New York
20	IT	Austin
30	Analytics	Boston
40	HR	Chicago

INNER JOIN Result				
id	name	department	location	salary
1	Alice Johnson	Engineering	New York	95000
2	Bob Smith	IT	Austin	65000
3	Carol White	Engineering	New York	120000
4	David Brown	Analytics	Boston	70000
5	Eva Green	IT	Austin	55000



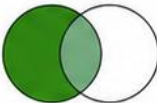
Only employees with a matching department are returned.

JOIN TYPES OVERVIEW



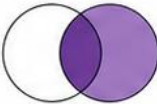
INNER JOIN

Returns only matching rows from both tables.



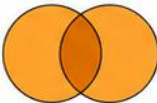
LEFT JOIN

Returns all rows from left table and matching rows from right table.



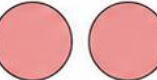
RIGHT JOIN

Returns all rows from right table and matching rows from left table.



FULL OUTER JOIN

Returns all rows from both tables, matched or unmatched.



CROSS JOIN

Returns the Cartesian product of both tables (all combinations).

INNER JOIN

An INNER JOIN returns only rows that have matching values in both tables.

```
SELECT c.full_name, a.account_number  
FROM customer c  
INNER JOIN account a  
    ON c.customer_id = a.customer_id;
```

Returns only customers who have at least one account -- customers with zero accounts are excluded.
The most commonly used join, and efficient when both join columns are indexed.
Best practice: index both sides of the join condition, e.g. account.customer_id.

KEY TAKEAWAY INNER JOIN is the default, most common join, and the one every other join type is defined relative to -- index both join columns to keep it fast.

INNER JOIN

Returns only the rows that have matching values in both tables based on the join condition.



KEY CONCEPTS

- **INNER JOIN** returns rows where there is a match in both tables.
- Uses the **ON** clause to define the join condition.
- Unmatched rows from either table are excluded.
- This is the most commonly used JOIN.
- Improves data accuracy by returning only related data.

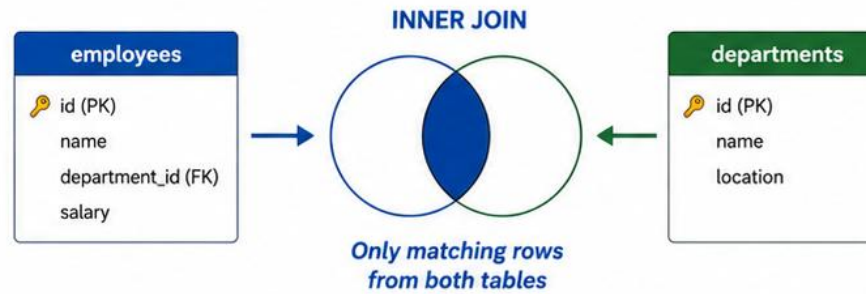


WHY IT MATTERS

- Helps combine related data for analysis and reporting.
- Reduces result set size by removing unrelated rows.
- Forms the foundation for building complex queries.



BEST PRACTICE: Always ensure the columns used in the JOIN condition are indexed for better performance.



SYNTAX AND EXAMPLE

```
SELECT e.id, e.name, e.salary, d.name AS department, d.location
FROM employees e
INNER JOIN departments d
ON e.department_id = d.id
ORDER BY e.name;
```

employees			
id	name	department_id	salary
1	Alice Johnson	10	95000
2	Bob Smith	20	65000
3	Carol White	10	120000
4	David Brown	30	70000
5	Eva Green	NULL	55000

departments		
id	name	location
10	Engineering	New York
20	IT	Austin
30	Analytics	Boston
40	HR	Chicago



INNER JOIN Result				
id	name	department	location	salary
1	Alice Johnson	Engineering	New York	95000
2	Bob Smith	IT	Austin	65000
3	Carol White	Engineering	New York	120000
4	David Brown	Analytics	Boston	70000



Row for Eva Green is excluded because there is no matching department (**NULL**).

HOW INNER JOIN WORKS

- 1 Compares rows from both tables using the join condition.
- 2 Returns only rows where the condition matches.
- 3 Excludes unmatched rows from both tables.

EXAMPLE USE CASES



Reporting employees with their departments.



Combining orders with customer details.



Retrieving products with category information.

LEFT JOIN

A LEFT JOIN returns every row from the left table, plus matching rows from the right -- unmatched right columns come back NULL.

```
SELECT c.full_name, a.account_number  
FROM customer c  
LEFT JOIN account a  
  ON c.customer_id = a.customer_id;
```

Amina Khan (has accounts) returns one row per account; Ravi Singh (zero accounts) returns one row with account_number = NULL.
Solves 'give me every customer, including ones with no accounts yet'.
Best practice: index the right table's join column, e.g. account.customer_id.

KEY TAKEAWAY LEFT JOIN solves the exact problem INNER JOIN can't: listing every customer, including ones who have no accounts yet, like Ravi Singh.

LEFT JOIN

Returns all rows from the left table and matching rows from the right table. If no match exists, **NULL** values are returned for the right table columns.



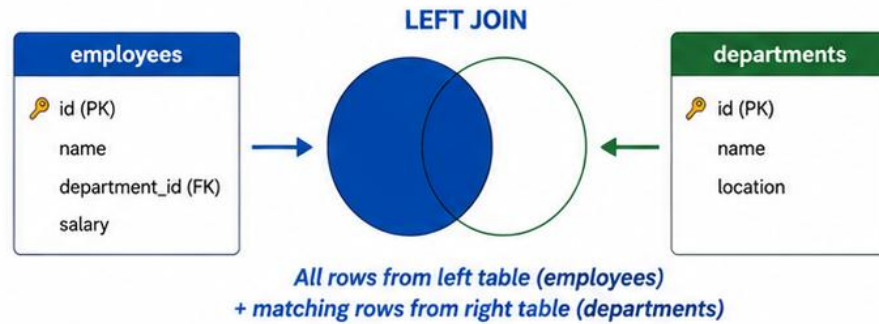
KEY CONCEPTS

- LEFT JOIN returns all rows from the left table.
- Matching rows from the right table are included.
- If no match is found, **NULL** is returned for right table columns.
- Useful when you want to keep all records from the primary (left) table.
- The opposite of RIGHT JOIN.



WHY IT MATTERS

- Ensures no data from the left table is lost.
- Helps identify missing relationships.
- Commonly used in reporting and dashboards.
- Supports data quality and completeness analysis.



SYNTAX AND EXAMPLE

```
SELECT e.id, e.name, e.salary, d.name AS department, d.location
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.id
ORDER BY e.name;
```

employees (Left Table)				departments (Right Table)			LEFT JOIN Result				
id	name	department_id	salary	id	name	location	id	name	department	location	salary
1	Alice Johnson	10	95000	10	Engineering	New York	1	Alice Johnson	Engineering	New York	95000
2	Bob Smith	20	65000	20	IT	Austin	2	Bob Smith	IT	Austin	65000
3	Carol White	30	120000	30	Analytics	Boston	3	Carol White	Analytics	Boston	120000
4	David Brown	NULL	70000	40	HR	Chicago	4	David Brown	NULL	NULL	70000
5	Eva Green	NULL	55000				5	Eva Green	NULL	NULL	55000



Employees David Brown and Eva Green have no matching department, so **NULL** values are returned for department and location.

HOW LEFT JOIN WORKS

- 1 Returns all rows from the left table (employees).
- 2 Matches rows from the right table (departments) when available.
- 3 If no match exists, **NULL** values are returned for right table columns.

EXAMPLE USE CASES



List all employees with their department information.



Find employees who are not yet assigned to a department.



Generate reports that must include all records from the main table.



BEST PRACTICE: Always use meaningful column names and index join columns to improve performance.



MULTIPLE JOINS

Real reports rarely need just two tables -- chain joins to pull data from three, four, or more tables in one query.

```
SELECT c.full_name, a.account_number, t.amount
FROM customer c
JOIN account a      ON a.customer_id = c.customer_id
JOIN transaction t  ON t.account_id  = a.account_id
WHERE c.status = 'ACTIVE';
```

Returns rows only where every JOIN condition matches -- a customer with accounts but zero transactions is dropped entirely from this result.
Join order in the FROM clause doesn't have to match execution order -- the planner reorders the joins internally based on cost.
Index every foreign key in the chain (account.customer_id, transaction.account_id), or each additional hop becomes a sequential scan.

KEY TAKEAWAY Chain joins left to right, one ON clause per join -- PostgreSQL's planner decides the actual join order internally, which is exactly what a plan's join nodes reveal.

Multiple Joins

Combine three or more tables to retrieve related data across multiple relationships.



KEY CONCEPTS

- Multiple joins allow you to combine data from several tables in a single query.
- Each JOIN connects tables using a related column.
- JOIN order and conditions define the result set.
- Use table aliases to keep queries readable.
- Ensure proper indexes on join columns for better performance.



WHY IT MATTERS

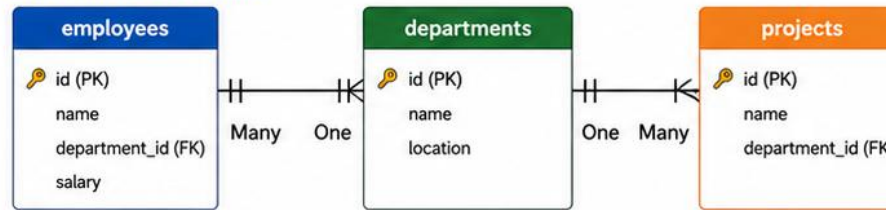
- Provides a complete view of related business data.
- Eliminates the need for multiple queries.
- Essential for reporting, analytics, and dashboards.
- Improves accuracy by combining normalized data.



BEST PRACTICE: Break complex queries into logical steps during development. Verify results at each JOIN stage.



EXAMPLE SCENARIO



Employees belong to a department, and departments have many projects.

JOIN TYPES USED

- **INNER JOIN:** Employees must have a department.
- **INNER JOIN:** Departments must have projects.



You can mix JOIN types based on business requirements.



TIPS

- Always qualify columns with table aliases.
- Use meaningful aliases for readability.
- Check join conditions carefully to avoid Cartesian products.
- Add indexes on foreign key columns.

SYNTAX AND EXAMPLE

```
SELECT e.id AS employee_id, e.name AS employee_name, e.salary,
       d.name AS department_name, d.location,
       p.name AS project_name
FROM   employees e
INNER JOIN departments d ON e.department_id = d.id
INNER JOIN projects p   ON d.id = p.department_id
ORDER BY e.name, p.name;
```

employees			
id	name	department_id	salary
1	Alice Johnson	10	95000
2	Bob Smith	20	65000
3	Carol White	10	120000
4	David Brown	30	70000
5	Eva Green	NULL	55000

departments		
id	name	location
10	Engineering	New York
20	IT	Austin
30	Analytics	Boston
40	HR	Chicago

projects		
id	name	department_id
100	Payroll System	10
101	Website Redesign	10
102	Mobile App	20
103	Data Warehouse	30



MULTIPLE JOIN RESULT				
employee_id	employee_name	department_name	location	project_name
1	Alice Johnson	Engineering	New York	Payroll System
1	Alice Johnson	Engineering	New York	Website Redesign
2	Bob Smith	IT	Austin	Mobile App
3	Carol White	Engineering	New York	Payroll System
3	Carol White	Engineering	New York	Website Redesign
4	David Brown	Analytics	Boston	Data Warehouse

SUBQUERIES

A subquery is a SELECT nested inside another SQL statement -- it can appear in WHERE, FROM, or SELECT itself.

```
SELECT full_name  
FROM customer  
WHERE customer_id IN (  
    SELECT customer_id FROM account WHERE balance > 10000  
);
```

The inner query runs independently of the outer query -- it doesn't reference outer columns.
Common uses: WHERE ... IN (subquery), or a subquery in FROM as a derived table.
Often rewritable as a JOIN -- compare readability and, later, the execution plan.

KEY TAKEAWAY A plain subquery executes independently -- WHERE customer_id IN (subquery) is a readable way to filter on a condition that itself requires a separate query to compute.

Subqueries

A **subquery** (nested query) is a query inside another SQL statement. It is used to return data that will be used by the outer query.



KEY CONCEPTS

- A subquery is a **SELECT** statement inside another query.
- The inner query executes first and returns a result to the outer query.
- Subqueries can be used in **WHERE**, **HAVING**, **FROM**, and **SELECT** clauses.
- They help simplify complex logic and reduce multiple steps.
- The subquery must return the expected number of rows and columns.



WHY IT MATTERS

- Makes complex queries easier to write and read.
- Allows data comparison and filtering.
- Reduces the need for joins in some cases.
- Essential for building advanced reporting queries.



BEST PRACTICE: Always review the execution plan for subqueries to ensure good performance.

EXAMPLE SCENARIO

Find employees who earn more than the average salary in their department.

employees				departments	
id	name	department_id	salary	id	name
1	Alice	10	95000	10	Engineering
2	Bob	10	65000	20	IT
3	Carol	20	120000	30	HR
4	David	20	70000		
5	Eva	20	55000		

One
to
Many

SUBQUERY IN HAVING CLAUSE

```
-- Departments with average salary greater
-- than overall company average
SELECT d.id, d.name, AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e ON d.id = e.department_id
GROUP BY d.id, d.name
HAVING AVG(e.salary) > (SELECT AVG(salary) FROM employees);
```

TYPES OF SUBQUERIES

	SINGLE-ROW SUBQUERY	Returns a single row (one value or one set of values).
	MULTI-ROW SUBQUERY	Returns multiple rows.
	CORRELATED SUBQUERY	Refers to columns from the outer query and is executed once per row.
	NON-CORRELATED SUBQUERY	Independent of the outer query and executed once.

SAMPLE RESULT (WHERE SUBQUERY)

id	name	department_id	salary
1	Alice	10	95000
3	Carol	20	120000

- ✓ Only employees whose salary is greater than their department's average are returned.

WHERE SUBQUERY EXAMPLE

```
-- Employees earning more than
-- their department average salary
SELECT e.id, e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = e.department_id
);
```



The subquery returns the average salary for each department. The outer query compares each employee's salary against it.

SUBQUERY IN FROM CLAUSE (DERIVED TABLE)

```
-- Top 3 highest paid employees
SELECT name, salary
FROM (
    SELECT name, salary
    FROM employees
    ORDER BY salary DESC
    LIMIT 3
) AS top_earners;
```

BEST PRACTICES

- ✓ Use subqueries for clarity, not when joins are simpler.
- ✓ Ensure the subquery returns the expected number of rows.
- ✓ Correlated subqueries can be slower—use carefully.
- ✓ Test subqueries independently for correctness.



CORRELATED SUBQUERIES

A correlated subquery references a column from the outer query -- conceptually, it re-runs once per outer row.

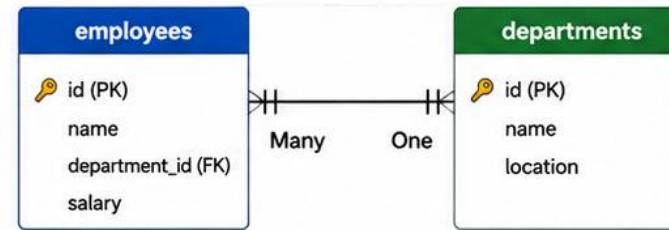
```
SELECT c.full_name
FROM customer c
WHERE EXISTS (
  SELECT 1 FROM account a
  WHERE a.customer_id = c.customer_id AND a.balance > 10000
);
```

a.customer_id = c.customer_id references the outer query's c -- that's what makes it correlated.
EXISTS stops at the first match per row -- often faster than IN for this shape.
PostgreSQL's planner can often rewrite a correlated subquery into a join internally.

KEY TAKEAWAY A correlated subquery references the outer query, so it's conceptually evaluated per outer row -- EXISTS is usually the most efficient way to write this pattern.

Correlated Subqueries

A correlated subquery refers to a column from the outer query and is executed once for each row processed by the outer query.



Employees belong to a department.



KEY CONCEPTS

- The inner query uses values from the outer query.
- Executed once per row of the outer query.
- Used when the result of the inner query depends on the current row.
- Commonly used for comparisons (EXISTS, COUNT, AVG, etc.).

EXAMPLE: Employees earning more than their department average salary

```
SELECT e.id, e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

Outer Query

Inner Query (correlated)



WHY IT MATTERS

- ✓ Powerful for row-by-row analysis.
- ✓ Avoids complex JOINS in some cases.
- ✓ Useful for conditional filtering.
- ✓ Essential for EXISTS and advanced logic.

employees			
id	name	department_id	salary
1	Alice	10	95000
2	Bob	10	65000
3	Carol	20	120000
4	David	20	70000
5	Eva	30	55000

departments	
id	name
10	Engineering
20	IT
30	HR

Department Averages

- Engineering (10): $(95000 + 65000) / 2 = 80000$
- IT (20): $(120000 + 70000) / 2 = 95000$
- HR (30): 55000



Key Idea

The inner query depends on the current row of the outer query.



Outer Query
Row



Inner Query
Uses That Row



Result for
That Row

HOW IT WORKS (Step-by-Step)

- 1 The outer query picks a row from employees.
- 2 The inner query runs using the department_id of that row.
- 3 It calculates the average salary for that department.
- 4 If the employee's salary is greater than the average, the row is returned.

Query Result

id	name	department_id	salary
1	Alice	10	95000
3	Carol	20	120000

- ✓ Only Alice and Carol earn more than their department average.



BEST PRACTICE: Use correlated subqueries when you need row-by-row logic that depends on the outer query.



CTES

A Common Table Expression (CTE), written with WITH, names a temporary result set scoped to one query -- often clearer than a nested subquery.

```
WITH high_value AS (  
    SELECT customer_id FROM account  
    GROUP BY customer_id HAVING SUM(balance) > 50000  
)  
SELECT c.full_name  
FROM customer c  
JOIN high_value h ON h.customer_id = c.customer_id;
```

high_value is evaluated once and referenced by name in the outer query, instead of repeating the same logic as a nested subquery.

A recursive CTE (WITH RECURSIVE) walks hierarchical data, such as an account ownership chain, one level per iteration.

A CTE doesn't automatically outperform an equivalent subquery -- check EXPLAIN to see whether PostgreSQL materializes or inlines it.

KEY TAKEAWAY A CTE names a step of a multi-step query so it reads top to bottom -- especially valuable once a query needs two or three layers of subquery logic.

CTEs

(Common Table Expressions)

A Common Table Expression (CTE) is a temporary named result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement. It improves readability and helps break complex queries into logical steps.



KEY CONCEPTS

- Defined using the **WITH** clause.
- Acts like a temporary, named result set.
- Exists only for the duration of the single query.
- Can be referenced one or multiple times.
- Improves readability and maintainability.
- Supports recursive queries for hierarchical data.



WHY IT MATTERS

- Simplifies complex queries by breaking them into smaller steps.
- Improves readability and reduces nesting.
- Reuse intermediate results without repeating subqueries.
- Easier to test and debug each step.



BEST PRACTICES



Use meaningful CTE names.



Keep CTEs small and focused.



Avoid using CTEs when a simple query or subquery is sufficient.



Check execution plans—CTEs may be materialized depending on the PostgreSQL version.



Index columns used in joins and filters for better performance.

EXAMPLE SCENARIO

Find the total salary paid in each department for employees earning more than 60,000.

employees			
id	name	department_id	salary
1	Alice	10	95000
2	Bob	10	65000
3	Carol	20	120000
4	David	20	55000
5	Eva	30	75000

departments	
id	name
10	Engineering
20	IT
30	HR

Many
to
One



- Step 1: Filter high earners using CTE.
- Step 2: Join with departments and aggregate.

HOW IT WORKS

- 1 The CTE (`high_earners`) filters employees with `salary > 60000`.
- 2 The main query joins the CTE result with departments.
- 3 Results are grouped by department and total salary is calculated.
- 4 Ordered by total salary in descending order.

CTE QUERY EXAMPLE

```
WITH high_earners AS (  
    SELECT id, name, department_id, salary  
    FROM employees  
    WHERE salary > 60000  
)  
SELECT d.name AS department_name,  
       SUM(h.salary) AS total_salary  
FROM high_earners h  
JOIN departments d  
    ON h.department_id = d.id  
GROUP BY d.name  
ORDER BY total_salary DESC;
```



The CTE (`high_earners`) is evaluated first, then used in the main query for joins and aggregation.

SAMPLE RESULT

department_name	total_salary
IT	120000
Engineering	160000
HR	75000

WHEN TO USE CTES



When a query has multiple logical steps.



When the same intermediate result is used multiple times.



For hierarchical or recursive data queries.



To make complex queries easy to read and maintain.

RECURSIVE CTE EXAMPLE (HIERARCHY)

Get all employees in a management hierarchy (starting from a manager).

```
WITH RECURSIVE org (id, name, manager_id) AS (  
    SELECT id, name, manager_id FROM employees WHERE id = 1  
    UNION ALL  
    SELECT e.id, e.name, e.manager_id  
    FROM employees e  
    JOIN org o ON e.manager_id = o.id  
)  
SELECT * FROM org;
```



AGGREGATIONS

Aggregate functions compute a single summary value across a group of rows -- COUNT, SUM, AVG, MIN, and MAX.

Function	Computes
COUNT(*)	Number of rows.
SUM(column)	Total of a numeric column.
AVG(column)	Average of a numeric column.
MIN(column) / MAX(column)	Smallest / largest value.

KEY TAKEAWAY Aggregate functions collapse many rows into one summary value -- combined with GROUP BY, covered next, they collapse many rows into one summary value per group.



Aggregations

Aggregations allow you to summarize and analyze data by calculating summary values across multiple rows.

Common Aggregate Functions



SUM(column)

Returns the total of numeric values.



COUNT(column)

Returns the number of rows (or non-null values).



AVG(column)

Returns the average of numeric values.



MIN(column)

Returns the smallest value.



MAX(column)

Returns the largest value.



Why Aggregations Matter

Aggregations help transform raw data into meaningful insights for reporting, dashboards, analytics, and decision-making.



Key Takeaway

Use aggregate functions to summarize data across multiple rows and reveal trends and patterns.

Example Table: orders

order_id	customer_id	amount	status
1	101	250.00	COMPLETED
2	102	150.00	COMPLETED
3	101	300.00	PENDING
4	103	200.00	COMPLETED
5	102	100.00	CANCELLED

Example: Aggregation Query

```
SELECT
  COUNT(*) AS total_orders,
  SUM(amount) AS total_amount,
  AVG(amount) AS average_amount,
  MIN(amount) AS min_amount,
  MAX(amount) AS max_amount
FROM orders
WHERE status = 'COMPLETED';
```

Result

total_orders	3
total_amount	600.00
average_amount	200.00
min_amount	150.00
max_amount	300.00

GROUP BY

GROUP BY collapses rows sharing the same value into groups, so an aggregate function computes one result per group.

```
SELECT status, COUNT(*) AS customer_count  
FROM customer  
GROUP BY status;
```

Every non-aggregated column in SELECT must appear in GROUP BY.

Groups rows by status first, then COUNT(*) counts rows within each group.

Common pairing: GROUP BY a foreign key to summarize a one-to-many relationship.

KEY TAKEAWAY GROUP BY status turns a flat customer list into a per-status summary -- one COUNT(*) row for PROSPECT, one for ACTIVE, one for each other status value present.



GROUP BY

GROUP BY groups rows that have the same values in one or more columns into summary rows, so aggregate functions can calculate results per group.

How GROUP BY Works



1. FROM / WHERE

Filter rows based on conditions.



2. GROUP BY

Group the remaining rows by the specified column(s).



3. AGGREGATE

Calculate summary values for each group.

Example Table: orders

order_id	customer_id	amount	status	order_date
1	101	250.00	COMPLETED	2024-05-01
2	102	150.00	COMPLETED	2024-05-01
3	101	300.00	PENDING	2024-05-02
4	103	200.00	COMPLETED	2024-05-02
5	102	100.00	CANCELLED	2024-05-03
6	101	150.00	COMPLETED	2024-05-03

Example: Total Amount per Customer

```
SELECT customer_id,  
       COUNT(*) AS total_orders,  
       SUM(amount) AS total_amount,  
       AVG(amount) AS average_amount  
FROM orders  
GROUP BY customer_id  
ORDER BY customer_id;
```

Query Result

customer_id	total_orders	total_amount	average_amount
101	3	700.00	233.33
102	2	250.00	125.00
103	1	200.00	200.00



Key Points

- All columns in **SELECT** that are not inside an aggregate function must appear in the **GROUP BY** clause.
- **GROUP BY** works with aggregate functions like **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**.
- **ORDER BY** is often used to sort the grouped results.



Use Cases

- Sales totals per customer, region, or product.
- Average order value per day or month.
- Counting items per category.

HAVING

HAVING filters groups after GROUP BY has aggregated them -- WHERE cannot reference an aggregate function's result.

```
SELECT customer_id, COUNT(*) AS account_count
FROM account
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

WHERE filters individual rows before grouping; HAVING filters groups after grouping.

HAVING COUNT(*) > 1 keeps only customers with more than one account.

A common mistake: writing WHERE COUNT(*) > 1, which PostgreSQL rejects.

KEY TAKEAWAY HAVING is WHERE's counterpart for groups -- it can reference an aggregate result like COUNT(*), which WHERE, evaluated before grouping happens, never can.

TRY IT YOURSELF — EXERCISE 1: WRITE THE REPORTING QUERIES

Reinforces: joins, subqueries, CTEs, aggregation, GROUP BY, and HAVING as you just covered them, written against the CRM schema.

STEPS (notes/lab38-queries.md)

Step	What To Do
1 — Inner vs Left	Write the join that lists customers with accounts, and the one that includes those without.
2 — Aggregate	Write a query counting accounts per customer using GROUP BY.
3 — HAVING	Add a filter keeping only customers with more than one account.
4 — CTE	Rewrite the aggregate as a CTE and say what that buys you.

Join and aggregate

```
inner join -> only customers that have accounts
left  join -> all customers; account columns null when none
group by c.id having count(a.id) > 1    -- WHERE filters rows, HAVING groups
```

TRY IT NOW — Complete Exercise 1 before continuing: `exercises/exercise-01-join-and-aggregate.md` (workspace: `examples/module-38-exercises`).



HAVING

HAVING filters groups after aggregation. It is used with **GROUP BY** to restrict groups based on aggregate conditions.

How HAVING Works



1. FROM / WHERE

Filter individual rows.



2. GROUP BY

Group rows by the specified column(s).



3. HAVING

Filter the grouped results based on aggregate conditions.



4. SELECT

Return the final aggregated results.

Example Table: orders

order_id	customer_id	amount	status	order_date
1	101	250.00	COMPLETED	2024-05-01
2	102	150.00	COMPLETED	2024-05-01
3	101	300.00	PENDING	2024-05-02
4	103	200.00	COMPLETED	2024-05-02
5	102	100.00	CANCELLED	2024-05-03
6	101	150.00	COMPLETED	2024-05-03

Example Query

```
SELECT customer_id,  
       COUNT(*) AS total_orders,  
       SUM(amount) AS total_amount,  
       AVG(amount) AS average_amount  
FROM orders  
GROUP BY customer_id  
HAVING SUM(amount) > 200  
ORDER BY total_amount DESC;
```

Query Result

customer_id	total_orders	total_amount	average_amount
101	3	700.00	233.33
102	2	250.00	125.00
103	1	200.00	200.00



Only groups with **total_amount** greater than 200 are included in the result.



Key Points

- **HAVING** is used with **GROUP BY**.
- **WHERE** filters rows before grouping; **HAVING** filters groups after grouping.
- Aggregate functions (SUM, COUNT, AVG, etc.) are typically used in **HAVING**.



Common Use Cases

- Find customers with total spending above a threshold.
- Find products with more than N orders.
- Identify categories with average order value above a target.

WINDOW FUNCTIONS OVERVIEW

A window function computes a value across a set of related rows without collapsing them into one row, using OVER().

```
SELECT customer_id, amount,  
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY created_at) AS running_total  
FROM transaction;
```

Unlike GROUP BY, every original row survives in the output -- nothing collapses.
PARTITION BY resets the calculation per group; ORDER BY controls running/ranking order.
Common uses: running totals, rankings (RANK, ROW_NUMBER), and moving averages.

KEY TAKEAWAY A window function adds a computed column alongside every original row -- PARTITION BY and ORDER BY inside OVER() control exactly how that calculation is scoped.



Window Functions Overview

Window functions perform calculations across a set of table rows that are somehow related to the current row.



Unlike aggregate functions (e.g., SUM, AVG) that collapse rows into a single result, window functions return a value for each row while considering a defined "window" of rows.

Example Table: sales

order_id	customer_id	order_date	amount
1	101	2024-05-01	120.00
2	101	2024-05-02	200.00
3	101	2024-05-03	150.00
4	102	2024-05-01	300.00
5	102	2024-05-02	250.00
6	103	2024-05-01	180.00

Common Window Functions

123

ROW_NUMBER()

Assigns a unique sequential number to rows within the window.



RANK()

Assigns a rank, with gaps for ties.



DENSE_RANK()

Assigns a rank, without gaps for ties.



SUM() OVER (...)

Calculates a running total or partitioned total.



AVG() OVER (...)

Calculates a running average or partitioned average.



LAG() / LEAD()

Accesses data from previous or next row within the window.

Example Query

```
SELECT
  order_id,
  customer_id,
  order_date,
  amount,
  ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY order_date
  ) AS row_num,
  SUM(amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND CURRENT ROW
  ) AS running_total
FROM sales
ORDER BY customer_id, order_date;
```



Query Result

order_id	customer_id	order_date	amount	row_num	running_total
1	101	2024-05-01	120.00	1	120.00
2	101	2024-05-02	200.00	2	320.00
3	101	2024-05-03	150.00	3	470.00
4	102	2024-05-01	300.00	1	300.00
5	102	2024-05-02	250.00	2	550.00
6	103	2024-05-01	180.00	1	180.00



Key Points

- Window functions operate on a set of rows related to the current row (the window).
- They do not reduce the number of rows in the result set.
- OVER clause defines how rows are partitioned and ordered.
- Ideal for running totals, rankings, moving averages, and comparisons.



Common Use Cases

- Running totals and moving averages
- Ranking customers, products, or sales
- Comparing current row with previous/next row
- Top-N per group (e.g., top 3 orders per customer)

OVER Clause Components

- PARTITION BY** – divides rows into partitions (like GROUP BY)
- ORDER BY** – defines the order within each partition
- ROWS/RANGE** – defines the window frame of rows

INDEX FUNDAMENTALS REVIEW

A quick recap before going deep: an index is a separate structure that lets PostgreSQL find rows without scanning the whole table.



Book-Index Analogy

Points straight to matching rows instead of reading every row.



B-Tree Default

PostgreSQL's default index type -- sorted, supports equality and range.



Speeds Reads, Costs Writes

Faster SELECT, slower INSERT/UPDATE/DELETE on that table.



Module 37 Foundation

Primary keys and foreign keys already have, or need, indexes - this module goes deeper.

KEY TAKEAWAY Module 37 introduced indexes; this module goes deep on choosing exactly which columns, in what order, and with what specialized form -- starting with selectivity, next.



Index Fundamentals Review

Indexes are special data structures that improve the speed of data retrieval by allowing the database to find rows without scanning the entire table.

Why Indexes Matter



Faster Reads

Index lookups are much faster than full table scans.



Better Performance

Improve query response time and overall throughput.



Efficient Sorting

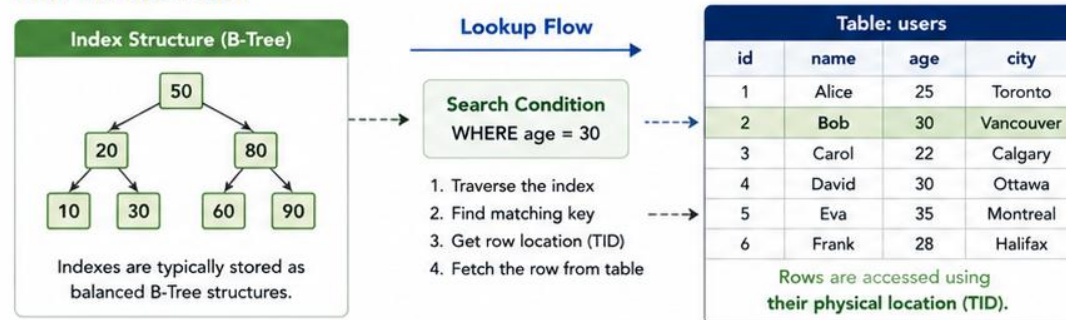
Help with ORDER BY, GROUP BY, and range queries.



Optimized Joins

Speed up join operations between tables.

How Indexes Work



Types of Indexes (Recap)



B-Tree Index

Default index type. Great for equality and range queries.



Unique Index

Enforces uniqueness of values in one or more columns.



Composite Index

Index on multiple columns in a defined order.



Partial Index

Indexes only rows that satisfy a specific condition.



Expression Index

Indexes the result of an expression or function.



GIN / GiST / BRIN

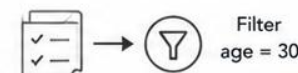
Specialized index types for JSON, full-text, geospatial, large data.

Example: Query With and Without Index

```
SELECT * FROM users WHERE age = 30;
```

Without Index

Seq Scan on users



- Scans every row
- Higher I/O and CPU
- Slower on large tables

With Index on (age)

Index Scan using idx_users_age



- Uses index to find matching keys
- Accesses only relevant rows
- Much faster for selective queries

Key Points

- ✓ Indexes speed up read operations but can slow down INSERT, UPDATE, DELETE.
- ✓ Use indexes on columns used in WHERE, JOIN, ORDER BY, and GROUP BY.
- ✓ Choose the right columns—highly selective columns work best.
- ✓ Too many indexes increase storage and maintenance cost.
- ✓ Always test and measure performance impact.

HOW POSTGRESQL USES INDEXES

PostgreSQL's planner decides, per query, whether using an index is actually cheaper than scanning the whole table.

An index existing does not guarantee PostgreSQL will use it -- the planner estimates cost both ways and picks the cheaper option. For a query matching most of a table, a full sequential scan can genuinely be cheaper than jumping through an index row by row. This is why EXPLAIN, covered later this module, is the only reliable way to confirm an index is actually being used. Fresh statistics (the ANALYZE command, covered later) are what make the planner's cost estimates trustworthy in the first place.

KEY TAKEAWAY Creating an index is a request, not a command -- PostgreSQL's planner only uses it when its own cost estimate says doing so is actually cheaper.



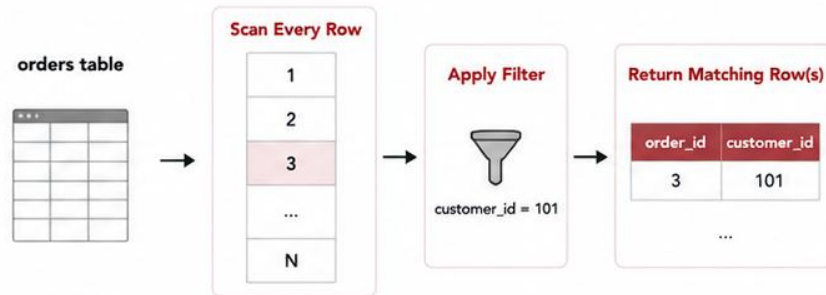
How PostgreSQL Uses Indexes

PostgreSQL uses indexes to avoid scanning entire tables. Instead, it quickly finds the matching rows by navigating the index and then accessing the required table rows.

Example Query

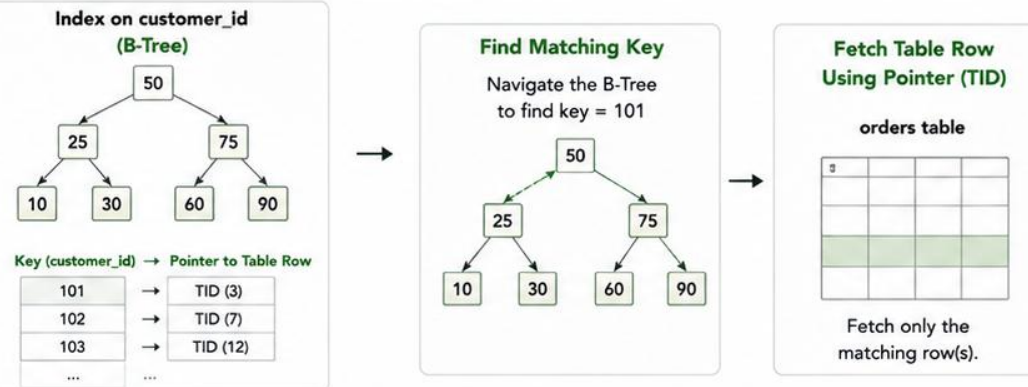
```
SELECT * FROM orders WHERE customer_id = 101;
```

Without Index (Sequential Scan)



- PostgreSQL reads every row in the table.
- Filtering happens after reading all rows.
- High I/O, CPU, and slower performance on large tables.

With Index (Index Scan)



- ✓ PostgreSQL uses the index to quickly find matching keys.
- ✓ It follows the pointer (TID: Tuple ID) to fetch only the required table row(s).
- ✓ Much less I/O and faster response time—especially on large tables.

When Will PostgreSQL Use an Index?



Equality Conditions
=, IN, and some
IS NULL queries.



Range Conditions
>, >=, <, <=,
BETWEEN.



**Pattern Matching
(With B-Tree)**
LIKE 'abc%'
(prefix search).



JOIN Operations
Indexes on join columns
speed up joins.



ORDER BY / GROUP BY
Indexes help avoid
explicit sorting.

Key Notes

- PostgreSQL chooses indexes based on cost estimation using statistics.
- Not all queries will use an index—sequential scan may be faster for small tables or low-selectivity conditions.
- Always review execution plans (EXPLAIN, EXPLAIN ANALYZE) to confirm index usage.

INDEX SELECTIVITY

Selectivity is the fraction of a table's rows a condition matches -- the lower the fraction, the more an index helps.

Column	Selectivity	Index Helps?
customer_id = 'CUS-1001'	1 row out of millions -- highly selective.	Yes, strongly.
status = 'ACTIVE'	Maybe 70% of all customers -- low selectivity.	Often not much.
is_deleted = false	Maybe 99% of all rows -- very low selectivity.	Rarely.

KEY TAKEAWAY A highly selective condition, one matching very few rows, is where an index pays off most -- a low-selectivity condition often performs better with a plain sequential scan.



Index Selectivity

Index selectivity measures how well an index can narrow down rows. High selectivity means the index can quickly find a small subset of rows. Low selectivity means many rows match, reducing the index's effectiveness.



Higher selectivity
= better performance

Lower selectivity
= less benefit from
the index

Example Table: orders (1,000,000 rows)

order_id (PK)	customer_id	status	order_date	amount
1	101	COMPLETED	2024-05-01	120.00
2	102	COMPLETED	2024-05-01	200.00
3	103	PENDING	2024-05-01	150.00
4	101	CANCELLED	2024-05-02	300.00
5	104	COMPLETED	2024-05-02	250.00
...	...	...	...	...

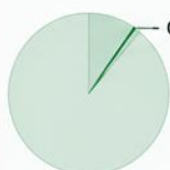
Total Rows: 1,000,000

High Selectivity (Good)



Column: customer_id (many unique values)

Index: idx_orders_customer_id



0.01%

Query Example

```
SELECT * FROM orders
WHERE customer_id = 101;
```

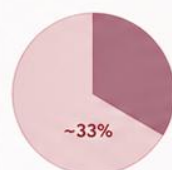
- ✓ Matches a very small percentage of rows (0.01% in this example)
- ✓ Index quickly locates the few matching rows
- ✓ High performance

Low Selectivity (Not Ideal)



Column: status (few distinct values)

Index: idx_orders_status



~33%

Query Example

```
SELECT * FROM orders
WHERE status = 'COMPLETED';
```

- ✗ Matches a large percentage of rows (~33%)
- ✗ Index still needs to fetch many rows
- ✗ May be slower than a sequential scan

Selectivity Formula

$$\text{Selectivity} = \frac{\text{Number of Distinct Values in Column}}{\text{Total Number of Rows in Table}}$$

Selectivity Examples

Column	Distinct Values	Total Rows	Selectivity	Assessment
customer_id	100,000	1,000,000	0.10 (10%)	High (Good)
order_date	1,095	1,000,000	0.0011 (0.11%)	Medium
status	3	1,000,000	0.000003 (0.0003%)	Low (Poor)
gender	2	1,000,000	0.000002 (0.0002%)	Very Low (Poor)

Higher selectivity values are better (closer to 1).

Factors Affecting Selectivity



Cardinality

Higher number of distinct values increases selectivity.



Data Distribution

Evenly distributed values improve selectivity.



Data Skew

Highly skewed data reduces effective selectivity.



Column Type

Date, numeric, and UUID columns are typically more selective.

Best Practices

- ✓ Create indexes on columns with high selectivity.
- ✓ Avoid indexing low-selectivity columns (e.g., status, gender, Y/N flags).
- ✓ Use composite indexes where the leading column has high selectivity.
- ✓ Regularly analyze statistics so PostgreSQL can make accurate decisions.



Check selectivity using statistics and EXPLAIN (ANALYZE).

COMPOSITE INDEXES

Revisiting composite indexes from Module 37 -- one index spanning multiple columns can serve several related query patterns.

```
CREATE INDEX idx_account_customer_status ON account (customer_id, status);
```

Serves WHERE customer_id = ? efficiently on its own -- the leading column.

Serves WHERE customer_id = ? AND status = ? even more efficiently -- both columns used.

Does NOT efficiently serve WHERE status = ? alone -- status is buried in second position.

KEY TAKEAWAY A composite index is sorted by its leading column first -- design column order from real query patterns, not alphabetically or by convention.



Composite Indexes

A composite (multi-column) index is an index on two or more columns. It can speed up queries that filter or sort using the leading column(s) of the index.



Column order matters.

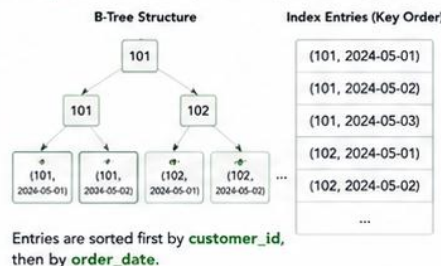
Put the most selective column first.

How It Works

Table: orders

order_id	customer_id	status	order_date	amount
1	101	COMPLETED	2024-05-01	120.00
2	101	COMPLETED	2024-05-02	200.00
3	102	COMPLETED	2024-05-01	150.00
4	102	PENDING	2024-05-02	300.00
5	101	CANCELLED	2024-05-03	250.00
...	...	...	...	...

Composite Index on (customer_id, order_date)



Create Composite Index

```
-- Create a composite index on (customer_id, order_date)
CREATE INDEX idx_orders_customer_date
ON orders (customer_id, order_date);
```

Equality + Range Example

```
SELECT * FROM orders
WHERE customer_id = 101
AND order_date BETWEEN '2024-05-01' AND '2024-05-31';
```

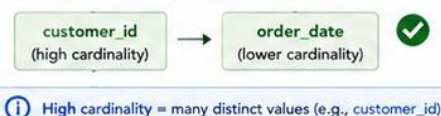
Highly efficient: equality on the first column, range on the second column.

Example Queries and Index Usage

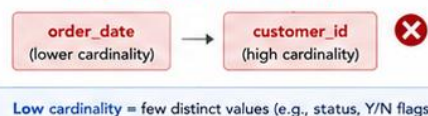
Query	Uses Index?	Why?
<pre>SELECT * FROM orders WHERE customer_id = 101 AND order_date >= '2024-05-01';</pre>	✓ Yes	Uses both columns; efficient range scan on order_date within customer_id = 101.
<pre>SELECT * FROM orders WHERE customer_id = 101;</pre>	✓ Yes	Uses the leading column (customer_id).
<pre>SELECT * FROM orders WHERE order_date = '2024-05-01';</pre>	✗ No	Cannot use index because leading column (customer_id) is not in the filter.
<pre>SELECT * FROM orders WHERE customer_id = 101 ORDER BY order_date;</pre>	✓ Yes	Index order matches ORDER BY; avoids sort.
<pre>SELECT * FROM orders WHERE customer_id = 101 AND status = 'COMPLETED';</pre>	🎯 Maybe	status is not part of the index; may filter after index scan.

Index Column Order Matters

Good Order (More Selective First)



Poor Order (Less Selective First)



Benefits of Composite Indexes



Faster Queries
Greatly improves queries with filters on multiple columns.



Reduces Sorting
Helps with ORDER BY clauses.



Supports Range Queries
Efficient for range conditions on the second column.



Covers Queries
Can make queries index-only if all needed columns are in the index.

Best Practices

- ✓ Put the most selective (high cardinality) column first.
- ✓ Use composite indexes for common query patterns.
- ✓ Avoid too many indexes; maintain only the ones you need.
- ✓ Monitor index usage with EXPLAIN and pg_stat_user_indexes.
- ✓ Keep index columns narrow (avoid wide text/blob columns).

Check Usage

```
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 101
AND order_date >= '2024-05-01';
```

Look for "Index Scan" using your composite index.



Composite indexes are powerful when used correctly, but the column order and query patterns determine their effectiveness.

INDEX COLUMN ORDER

The leftmost-prefix rule: a composite index efficiently serves any query filtering on a leading, unbroken run of its columns.

Index	Efficiently Serves	Does Not Serve
(a, b, c)	WHERE a=?, WHERE a=? AND b=?, WHERE a=? AND b=? AND c=?	WHERE b=? alone, WHERE c=? alone
(status, created_at)	WHERE status=?, WHERE status=? AND created_at>?	WHERE created_at>? alone

KEY TAKEAWAY Column order is not arbitrary -- put the column your queries filter on most often, alone or in combination, first.



Index Column Order

In composite indexes, the order of columns matters. PostgreSQL can efficiently use an index starting from the leftmost column and continuing to the right.



Key Principle

Place the **most selective** (highest cardinality) column first, followed by columns commonly used in query filters.



Example Table

orders

customer_id	order_date	status	amount
101	2024-05-01	SHIPPED	250.00
101	2024-05-02	PENDING	125.00
102	2024-05-01	SHIPPED	320.00
103	2024-05-03	CANCELLED	75.00



Why it matters

Wrong column order can lead to index scans or even sequential scans, hurting performance.



How Index Column Order Works

PostgreSQL uses the index from left to right. It can use a prefix of the index, but stops when a gap is encountered.



Good Index Order

```
CREATE INDEX idx_orders_cust_date  
ON orders (customer_id, order_date);
```

Query Pattern	Index Used?
WHERE customer_id = 101	✓ Yes
WHERE customer_id = 101 AND order_date >= '2024-05-01'	✓ Yes
WHERE order_date >= '2024-05-01'	⚡ No (Starts from first column)
WHERE status = 'SHIPPED'	✗ No (Not in index)



Bad Index Order

```
CREATE INDEX idx_orders_date_cust  
ON orders (order_date, customer_id);
```

Query Pattern	Index Used?
WHERE customer_id = 101	⚡ No (First column not used)
WHERE customer_id = 101 AND order_date >= '2024-05-01'	⚡ Partial (Only order_date used)
WHERE order_date >= '2024-05-01'	✓ Yes
WHERE status = 'SHIPPED'	✗ No (Not in index)



Best Practices

- Place equality columns first, then range columns.
- Put the most selective column first.
- Avoid low-cardinality columns (e.g., status) at the beginning.
- Design indexes based on your most common query patterns.



Well-designed index column order = faster queries, lower I/O, better performance.



PARTIAL INDEXES

A partial index adds a WHERE clause to CREATE INDEX itself -- indexing only the subset of rows a query pattern actually needs.

```
CREATE INDEX idx_account_open ON account (customer_id)
  WHERE closed_at IS NULL;
```

Only indexes open accounts -- closed accounts are excluded from the index entirely.
Smaller index, faster to maintain, and a strong match for a genuinely common query pattern like 'find this customer's open accounts'.
The query's own WHERE clause must be provably compatible with the index's WHERE clause.

KEY TAKEAWAY A partial index only indexes the rows a specific, common query pattern actually needs -- smaller, faster to maintain, and often a better fit than indexing the whole column.



Partial Indexes

A partial index indexes only a subset of rows that meet a specified condition. It reduces index size and improves performance for common filtered queries.



Example Scenario

We have an orders table where most queries fetch only active (status = 'ACTIVE') orders. Instead of indexing the entire table, we index only active orders.

Table: orders

order_id	customer_id	status	order_date	amount
1	101	ACTIVE	2024-05-01	250.00
2	102	CANCELLED	2024-05-02	125.00
3	103	ACTIVE	2024-05-02	320.00
4	104	PENDING	2024-05-03	150.00
5	105	ACTIVE	2024-05-03	275.00
6	106	CANCELLED	2024-05-04	90.00
...	...	...	...	...



Benefits

- Smaller index size
- Faster queries for filtered data
- Less maintenance overhead (faster updates/inserts)



Creating a Partial Index

```
CREATE INDEX idx_orders_active  
ON orders (customer_id, order_date)  
WHERE status = 'ACTIVE';
```



How It Works



Only rows where status = 'ACTIVE' are included in the index.



The index is smaller and faster to scan.



Queries with the same condition can use the index.



Queries not matching the condition will not use this index.



When to Use

- When queries frequently filter on a specific condition.
- When only a small portion of rows need to be indexed.
- To improve performance and reduce storage.



Query Example

Query that can use the partial index:

```
SELECT *  
FROM orders  
WHERE status = 'ACTIVE'  
AND customer_id = 101;
```



Uses partial index

Query that cannot use the partial index:

```
SELECT *  
FROM orders  
WHERE status = 'CANCELLED';
```



Cannot use partial index



Best Practices

- Use partial indexes for columns with low cardinality values that are frequently filtered.
- Ensure the query condition matches the index predicate.
- Do not overuse—each index adds write overhead.
- Monitor usage with `EXPLAIN ANALYZE`.



Partial indexes index only what you need—faster queries, smaller indexes, better performance.



EXPRESSION INDEXES

An expression index indexes the result of a function or expression, not just a raw column -- letting a transformed search stay fast.

```
CREATE INDEX idx_customer_lower_email ON customer (LOWER(email));
```

Matches a query written WHERE LOWER(email) = LOWER(?) -- the exact same expression.

Solves the 'function on an indexed column defeats the index' problem, covered later this module, for cases that genuinely need the transformation.

Any deterministic expression works, not just LOWER() -- e.g. date_trunc('day', created_at).

KEY TAKEAWAY An expression index lets a case-insensitive or otherwise transformed search stay index-backed, by indexing the transformed value directly instead of the raw column.

TRY IT YOURSELF — EXERCISE 2: REASON ABOUT INDEX SELECTIVITY

Reinforces: how PostgreSQL uses indexes, selectivity, composite column order, partial and expression indexes as you just covered.

STEPS (notes/lab38-index-selectivity.md)

Step	What To Do
1 — Selectivity	Say why an index on a boolean column is usually useless.
2 — Column Order	Given a filter on customer_id and a sort on opened_at, order the composite.
3 — Partial Index	Give a CRM case where a partial index beats a full one.
4 — Expression Index	Give the index that makes a case-insensitive email lookup fast.

Index selection

```
low selectivity (is_active) -> planner prefers a sequential scan
index account(customer_id, opened_at desc) -- filter first, then sort
partial:      where status = 'ACTIVE'      expression: lower(email)
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-index-selectivity.md (workspace: examples/module-38-exercises).



Expression Indexes

An expression index indexes the result of an expression instead of a column. It helps optimize queries that filter or sort using functions or calculations.



Example Scenario

We search users by email in a case-insensitive way. Without an index, PostgreSQL must apply LOWER(email) to every row, preventing normal index usage.

Table: users

id	email	created_at
1	Alice@example.com	2024-05-01
2	bob@example.com	2024-05-02
3	CHARLIE@example.com	2024-05-03
4	david@example.com	2024-05-04
...	...	...



Benefits

- ✓ Speeds up queries using functions or expressions.
- ✓ Allows indexing without storing extra computed columns.
- ✓ Smaller than indexing large/complex computed values elsewhere.
- ✓ Transparent to application code.



When to Use

- When queries use functions or expressions in WHERE or ORDER BY clauses.
- When you cannot change the query to avoid function calls.
- To improve performance without altering application logic.



Creating an Expression Index

```
CREATE INDEX idx_users_lower_email
ON users (LOWER(email));
-- Indexes the result of LOWER(email)
```



How It Works



The index stores the expression result (e.g., LOWER(email)).



When the query uses the same expression, PostgreSQL can use the index.



Greatly reduces the number of rows scanned.



Important: The query must use the exact same expression for the index to be used.



Query Examples



Uses the expression index

```
SELECT *
FROM users
WHERE LOWER(email) = 'alice@example.com';
```



Does NOT use the expression index

```
SELECT *
FROM users
WHERE email = 'alice@example.com';
-- Function not used → index cannot be used
```

Execution Plan Comparison



With Expression Index

```
Index Scan using idx_users_lower_email on users (cost=0.28..8.30
rows=4 width=...)
Index Cond: (lower(email) = 'alice@example.com'::text)
```



Without Expression Index

```
Seq Scan on users (cost=0.00..35.50 rows=1 width=...)
Filter: (lower(email) = 'alice@example.com'::text)
```



Best Practices

- Use for frequently used expressions in filters or sorts.
- Keep expressions simple and deterministic.
- Avoid non-immutable functions (e.g., now(), random()).
- Monitor usage with EXPLAIN ANALYZE.



Expression indexes index how your data is used, not just what it is. Use them to make function-based queries fast and efficient.



QUERY PLANNER OVERVIEW

The query planner is the PostgreSQL component that turns a parsed SQL statement into an actual execution strategy.



Considers Multiple Plans

Generates several candidate strategies for the same query -- different scans, different join orders.



Estimates Cost

Assigns each candidate an estimated cost, in arbitrary planner units, based on statistics.



Picks the Cheapest

Executes whichever candidate plan it estimates will cost the least.



Depends on Statistics

Cost estimates are only as good as the table statistics ANALYZE maintains.

KEY TAKEAWAY The planner doesn't guess -- it generates real alternative strategies, estimates each one's cost from table statistics, and executes the cheapest -- which is exactly what EXPLAIN reveals.



Query Planner Overview

The PostgreSQL query planner analyzes your SQL statement and chooses the most efficient execution plan to retrieve the data. It aims to minimize the total cost of execution.

How the Query Planner Works



1. Parse

Analyzes the SQL and checks syntax.



2. Rewrite

Rewrites the query into a standard form (Query Tree).



3. Plan

Considers multiple execution strategies and estimates cost.



4. Choose Plan

Selects the plan with the lowest estimated cost.



5. Execute

Executes the plan and returns results.



Key Points

- PostgreSQL uses a cost-based optimizer.
- Lower cost ≠ actual time, but generally better plan.
- Good statistics lead to better plans. Run **ANALYZE** regularly.
- The planner does not execute the query – it only creates the plan.
- Use **EXPLAIN** and **EXPLAIN ANALYZE** to see the plan.



Factors Influencing the Plan



Table & index statistics



Query structure & predicates



Available indexes & constraints



System settings (e.g., work_mem)



Data volume & distribution



Goals of the Query Planner

- Understand the query and its operations
- Estimate the cost of different execution strategies
- Choose the plan with the lowest estimated cost
- Execute the query using the chosen plan

Example

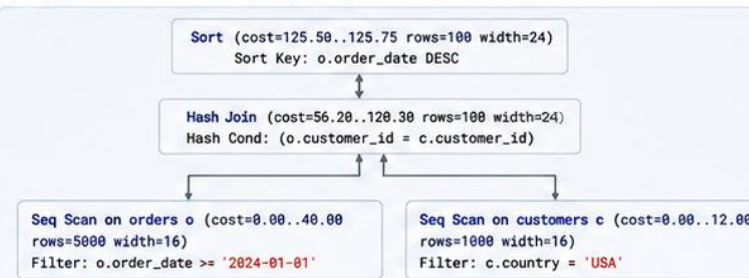
```
SELECT o.order_id, c.name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'USA'
AND o.order_date >= '2024-01-01'
ORDER BY o.order_date DESC;
```



Planner uses statistics

- Table row counts
- Index availability
- Data distribution
- Column statistics
- Most common values
- Correlation

Sample Execution Plan (simplified)



Plan Node Types

- Sort** Sorts result rows
- Hash Join** Joins using hash algorithm
- Seq Scan** Sequential scan of a table
- Index Scan** Uses an index



Best Practices

- Keep statistics up to date (**ANALYZE**).
- Use appropriate indexes.
- Avoid functions on indexed columns in **WHERE**.
- Write sargable (index-friendly) predicates.
- Review execution plans for slow queries.



The query planner chooses not just how to execute your query, but also how to join, filter, sort, and access your data.



POSTGRESQL COST-BASED OPTIMIZER

PostgreSQL's optimizer assigns every plan a cost using two components: startup cost and total cost.

Cost Component	Meaning
Startup Cost	Estimated cost before the first row can be returned, e.g. building a hash table before a Hash Join can begin probing it.
Total Cost	Estimated cost to return every row of this operation's output.
Row Estimate	How many rows this operation is expected to produce -- feeds the cost estimate for whatever operation is above it in the plan.

KEY TAKEAWAY Cost is a unitless estimate, not milliseconds -- what matters is comparing candidate plans against each other, which is exactly what the optimizer does automatically.



PostgreSQL Cost-Based Optimizer

PostgreSQL uses a cost-based optimizer (CBO) to choose the most efficient execution plan. It estimates the cost of alternative plans and selects the one with the lowest estimated cost.



How the Cost-Based Optimizer Works



1. Parse

Parse and validate the SQL statement.



2. Collect Statistics

Use table and index statistics (ANALYZE) to understand data distribution.



3. Generate Plans

Create multiple possible execution plans using different join orders and access paths.



4. Estimate Costs

Estimate the cost of each plan using the cost model.



5. Choose Best Plan

Select the plan with the lowest estimated cost.



6. Execute

Execute the chosen plan and return results.



Goals of the Cost-Based Optimizer

- Minimize the total cost of executing a query
- Reduce I/O, CPU usage, and memory consumption
- Choose the fastest plan based on available statistics



What Determines the Cost?

- I/O cost (disk reads/writes)
- CPU cost (processing rows)
- Memory usage
- Number of rows processed
- Row width (size)
- Index selectivity
- Join algorithms
- Sort operations
- Parallelism
- System configuration



Lower cost ≠ actual time, but usually correlates with better performance. Keep statistics up to date for accurate estimates (ANALYZE).



Sources of Statistics

- Table statistics: row count, size
- Column statistics: distinct values, most common values, null fraction, histogram
- Index statistics: depth, leaf pages, correlation
- Updated by: ANALYZE

```
ANALYZE;  
ANALYZE orders (customer_id, order_date);
```

Example: Alternative Plans for a Query

```
SELECT * FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
WHERE c.country = 'USA' AND o.order_date >= '2024-01-01';
```

Plan A

Nested Loop Join
Using Index Scan
Cost: 125.50
Rows: 1,000



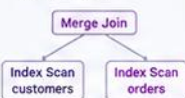
Plan B

Hash Join
Seq Scan on orders
Cost: 210.75
Rows: 1,000



Plan C

Merge Join
Using Index Scan
Cost: 180.25
Rows: 1,000



Chosen Plan

Plan A (Lowest Cost)
Cost: 125.50



Key Takeaways

- The optimizer compares many possible plans.
- It uses statistics and cost estimates, not guesswork.
- Good statistics lead to good plans.
- Run ANALYZE regularly.
- You can view and compare plans using EXPLAIN and EXPLAIN ANALYZE.



Cost Model Configuration

PostgreSQL uses configurable cost constants:

```
seq_page_cost      = 1.0  
random_page_cost   = 4.0  
cpu_tuple_cost     = 0.01  
cpu_index_tuple_cost = 0.005  
cpu_operator_cost  = 0.0025
```

These values influence plan choices.
Adjust only with expert guidance.



Best Practices

- Keep statistics current.
- Design appropriate indexes.
- Write sargable (index-friendly) queries.
- Review execution plans for slow queries.



The cost-based optimizer is the brain behind PostgreSQL's performance. Help it with good statistics, proper indexing, and well-written queries.



EXPLAIN

EXPLAIN shows a query's chosen execution plan and its estimated costs -- without actually running the query.

```
EXPLAIN SELECT * FROM customer WHERE status = 'ACTIVE';
```

```
Seq Scan on customer (cost=0.00..18.20 rows=140 width=64)  
  Filter: (status = 'ACTIVE'::text)
```

Read plans bottom-up: the Seq Scan runs first, and its output feeds the node above it.

cost=0.00..18.20 is startup cost and total cost in arbitrary planner units, not seconds -- useful for comparing plans, not as wall-clock time.

rows=140 is an estimate from table statistics, not a measured count -- EXPLAIN alone never actually runs the query.

KEY TAKEAWAY EXPLAIN reveals the planner's chosen strategy and its cost/row estimates -- it never executes the query, so it's always safe to run, even against a slow or destructive statement.

EXPLAIN

Understand How PostgreSQL Executes Your Queries



PostgreSQL

What is EXPLAIN?

The EXPLAIN command shows you the query execution plan that PostgreSQL will use to run your SQL statement. It helps you understand how the database accesses data, what indexes are used, and which operations are performed.

Example

```
EXPLAIN SELECT e.id, e.name, d.name AS department
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE e.status = 'ACTIVE';
```

Sample Query Plan Output

```
EXPLAIN SELECT e.id, e.name, d.name AS department
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE e.status = 'ACTIVE';

QUERY PLAN
-----
Hash Join  (cost=12.34..45.67 rows=100 width=72)
  Hash Cond: (e.department_id = d.id)
    -> Seq Scan on employees e  (cost=0.00..23.45 rows=500 width=36)
        Filter: (status = 'ACTIVE'::text)
    -> Hash  (cost=8.76..8.76 rows=100 width=36)
        -> Seq Scan on departments d  (cost=0.00..8.76 rows=100 width=36)
```

Operation

The type of operation
(e.g., Hash Join, Seq Scan).

Cost

Estimated cost (lower
is generally better).

Rows

Estimated number
of rows returned.

Width

Estimated row size
in bytes.



Key Takeaway

Use EXPLAIN (and EXPLAIN ANALYZE) to identify performance bottlenecks, choose the right indexes, and write more efficient queries.

EXPLAIN ANALYZE

EXPLAIN ANALYZE actually runs the query and reports real elapsed time and real row counts alongside the plan.

```
EXPLAIN ANALYZE SELECT * FROM customer WHERE status = 'ACTIVE';
```

```
Seq Scan on customer (cost=0.00..18.20 rows=140 width=64)  
  (actual time=0.012..0.041 rows=138 loops=1)  
    Filter: (status = 'ACTIVE'::text)
```

actual time=0.012..0.041 is measured in milliseconds -- time to the first row, then time to return all rows.
rows=138 actual versus rows=140 estimated shows how close the planner's statistics are to the real data.
loops=1 means this node ran once; a nested loop's inner scan can report loops far greater than 1.

KEY TAKEAWAY EXPLAIN ANALYZE actually executes the query -- the actual time and actual rows values are real measurements, and comparing them to the estimates is the single most useful diagnostic habit this module teaches.



`EXPLAIN ANALYZE`

`EXPLAIN ANALYZE` runs the query and shows the actual execution plan with real performance metrics.



Syntax:

```
EXPLAIN ANALYZE [VERBOSE] [BUFFERS] your_sql_query;
```

Example

Analyze a query that fetches active employees by department:

```
EXPLAIN ANALYZE SELECT * FROM employees WHERE department_id = 10 AND active = true;
                                QUERY PLAN
-----
Index Scan using idx_emp_dept_active on employees (cost=0.28..12.45 rows=25 width=128)
  Index Cond: ((department_id = 10) AND (active = true))
Planning Time: 0.142 ms
Execution Time: 0.356 ms
(4 rows)
```



Key Takeaway: `EXPLAIN ANALYZE` is your most powerful tool for understanding real query performance and making data-driven optimization decisions.

Why Use `EXPLAIN ANALYZE`?



Shows actual execution time
Reveals real-time performance of each step.



Displays the execution plan
Understand how PostgreSQL executes your query.



Includes row counts
See how many rows were processed at each step.



Helps identify bottlenecks
Pinpoint slow operations and inefficient scans or joins.



Guides performance tuning
Use insights to improve indexes, queries, and table design.

READING EXECUTION PLANS

A plan is a tree -- read it from the innermost, most-indented operations outward to see the true order of execution.

Indentation shows nesting: an indented line is an input feeding the operation above it.
Execution actually happens bottom-up and inside-out, even though the text reads top-down.
Compare estimated vs. actual rows at every node, not just the top line -- the divergence can start several levels deep.
A large gap between estimated and actual rows, or an unexpected Seq Scan, are the two signals worth investigating first.

KEY TAKEAWAY Read a plan from the most-indented, innermost operations outward -- that's the true order PostgreSQL actually executes in, even though the text lists the outermost operation first.



Reading Execution Plans

PostgreSQL shows how a query will be executed. Understanding the plan helps you find bottlenecks and optimize performance.



Example Query

```
SELECT o.order_id, c.name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'USA'
AND o.order_date >= '2024-01-01'
ORDER BY o.order_date DESC;
```



Key Plan Metrics

Cost: Estimated cost to retrieve rows from this node.
Rows: Estimated number of rows returned by this node.
Actual Rows: Actual rows returned (with ANALYZE).
Time: Actual time to execute this node (with ANALYZE).
Width: Average row size in bytes.

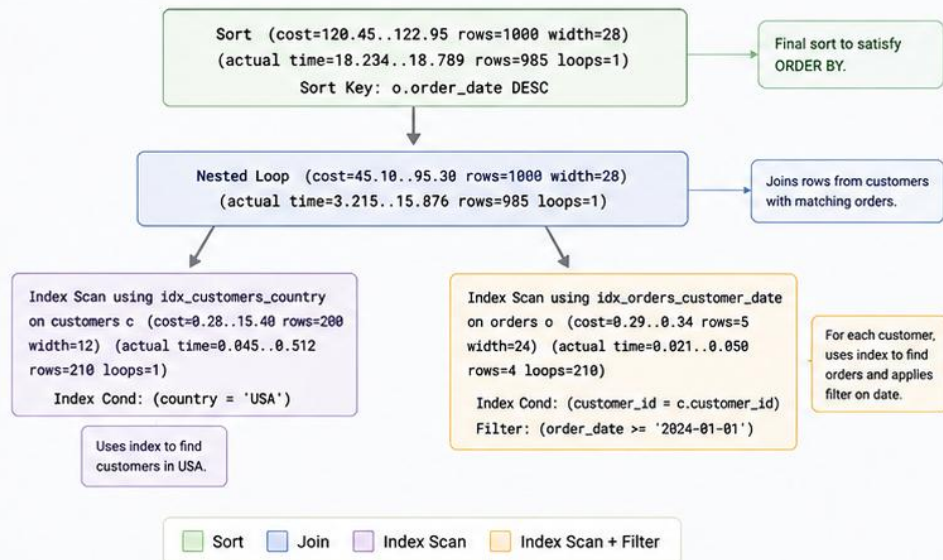


Common Node Types

Seq Scan	Sequentially scans the entire table.
Index Scan	Uses an index to find matching rows.
Index Only Scan	Uses index without reading table.
Bitmap Heap Scan	Uses index bitmap then heap.
Nested Loop	Loops over rows from one table.
Hash Join	Joins using a hash table.
Merge Join	Joins using sorted inputs.
Sort	Sorts rows.



Sample Execution Plan (EXPLAIN ANALYZE)



What to Look For



High Cost / Time

Focus on nodes with highest actual time.



Large Row Counts

More rows = more processing and I/O.



Seq Scan on Large Tables

May indicate missing or unusable index.



Sorts or Hashes

Can be expensive on large datasets.



Loops (Nested Loop)

High loop counts can multiply work.



How to Read a Plan

- ✓ Start from the top node: this is the final result.
- ✓ Follow the arrows downward (top-down).
- ✓ Understand each node and what it does.
- ✓ Check the estimated cost and actual time.
- ✓ Look for expensive nodes and row counts.
- ✓ Compare estimated rows vs actual rows.



Estimated vs Actual

Big differences can indicate:

- Outdated statistics
- Bad estimates from the planner
- Data skew or distribution issues

Run **ANALYZE** to update statistics.

Example Plan Table (Summary)

Node	Cost	Rows	Actual Rows	Actual Time (ms)
Sort	120.45	1000	985	18.789
Nested Loop	45.10	1000	985	15.876
Index Scan (customers)	0.28	200	210	0.512
Index Scan (orders)	0.29	5	4	0.050



Best Practices

- Keep statistics up to date: **ANALYZE** regularly.
- Create appropriate indexes.
- Avoid functions on indexed columns.
- Review execution plans for slow queries.
- Test changes and compare plans.



Execution plans are your roadmap—read them, understand them, and use them to make queries fast.



SCAN TYPES: SEQUENTIAL, INDEX, INDEX ONLY, BITMAP HEAP

Four scan node types cover almost every way PostgreSQL actually reads rows from a table.



Sequential Scan

Reads every row in table order. Cheap per-row, but cost grows with table size -- fine for small tables or low-selectivity filters.



Index Scan

Uses an index to find matching rows, then fetches each one from the table. Ideal for highly selective conditions.



Index Only Scan

Answers entirely from the index itself, with no table fetch -- possible only when every selected column is in the index.



Bitmap Heap Scan

Builds a bitmap of matching pages from an index, then reads those pages together -- a middle ground for moderate selectivity.

KEY TAKEAWAY Sequential Scan reads everything; Index Scan and Index Only Scan use an index directly; Bitmap Heap Scan is the planner's middle-ground choice for moderately selective conditions.

Sequential Scan



A **Sequential Scan** reads every row in a table from start to finish to find the rows that match the query.

Key Points

- PostgreSQL scans the entire table page by page.
- Used when no suitable index exists or when the query returns a large portion of the table.
- More I/O and CPU cost compared to an Index Scan.
- Common on small tables or low-selectivity queries.



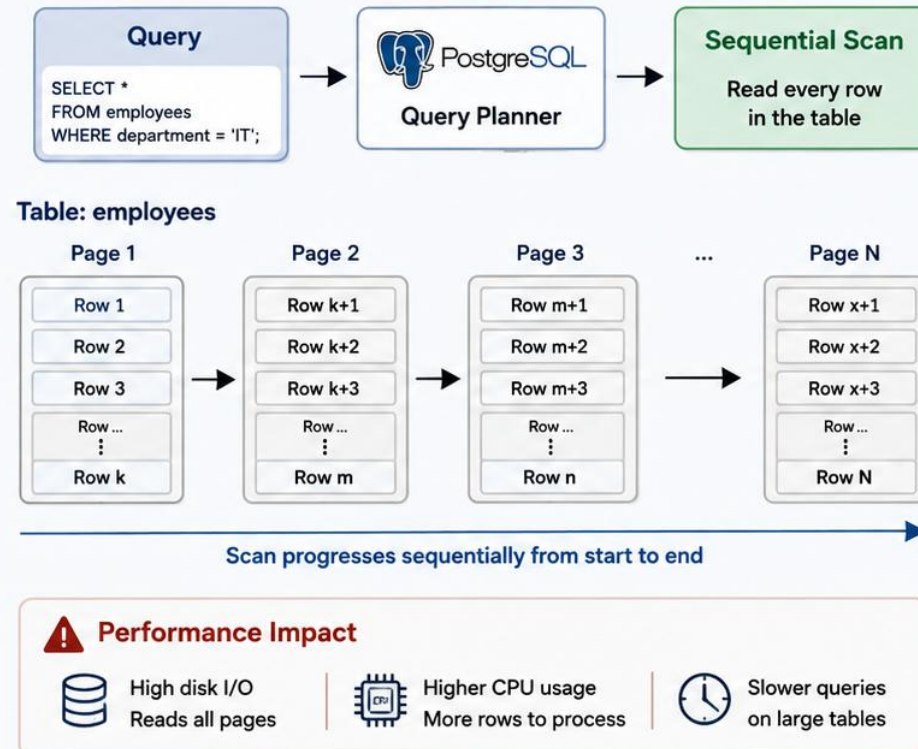
When is it OK?

Sequential scans are efficient on small tables or when a large percentage of rows must be returned.



Goal: Avoid unnecessary sequential scans by using appropriate indexes and writing selective queries.

How a Sequential Scan Works



Index Scan



An **Index Scan** uses an index to quickly locate the rows that match the query instead of reading the entire table.

Key Points

- PostgreSQL uses the index to find matching keys.
- Then it retrieves only the required table rows.
- Much less I/O and CPU compared to Sequential Scan.
- Best for selective queries that return a small percentage of rows.
- Performance depends on index quality and selectivity.



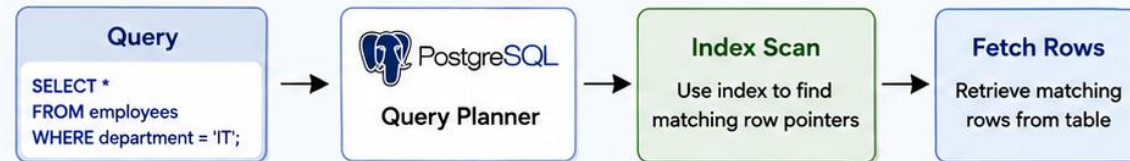
When is it OK?

Index scans are ideal when:
An index exists on the filtered column(s) and the query returns a small number of rows.



Goal: Use indexes to minimize data scanned and improve query performance.

How an Index Scan Works



Example

Index on department
(B-Tree Index)

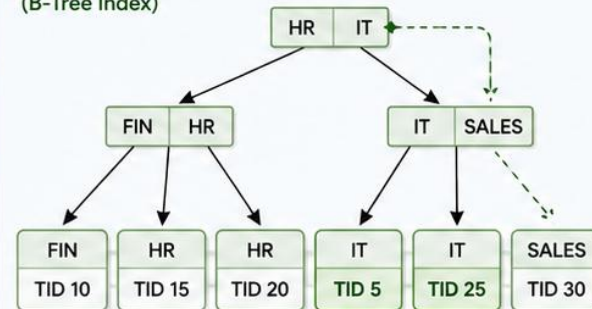


Table: employees

TID (Row Pointer)	id	name	department
5	101	Alice	IT
10	102	Bob	FIN
15	103	Carol	HR
20	104	David	HR
25	105	Eve	IT
30	106	Frank	SALES
...	...	...	...

Only matching rows (IT department) are fetched from the table.

Performance Benefits



Less I/O
Reads only necessary rows



Lower CPU Usage
Processes fewer rows



Faster Queries
Ideal for selective searches

Index Only Scan



An **Index Only Scan** retrieves all the required data directly from the index without accessing the table.

Key Points

- PostgreSQL uses the index to find the matching entries.
- If the index contains all the needed columns, it returns the data without visiting the table (heap).
- Avoids heap fetches, resulting in less I/O and CPU usage.
- Requires a covering index that includes all requested columns.
- Effectiveness depends on visibility map being set (heap pages marked all-visible).



When is it OK?

When a covering index exists and the visibility map indicates the rows are all-visible. Ideal for read-mostly workloads and reporting queries.



Goal: Use Index Only Scans to maximize performance by avoiding unnecessary table lookups.

How an Index Only Scan Works



Example

Query: `SELECT id, name FROM employees WHERE department = 'IT';`
Index: `idx_emp_dept_id_name ON employees (department, id, name)`

Index (covers all needed columns)			
department	id	name	(TID)
FIN	102	Bob	(10)
HR	103	Carol	(15)
IT	101	Alice	(5)
IT	105	Eve	(25)
IT	107	Grace	(40)
...	...	...	...

All required columns
(id, name) are in the index.
No need to access
the table.

Visibility Map
Pages are marked
all-visible

No heap
access

Table: employees (Heap)

id	name	department	salary
101	Alice	IT	70000
102	Bob	FIN	65000
103	Carol	HR	62000
104	David	HR	60000
105	Eve	IT	72000
106	Frank	SALES	58000
...	...	...	...

✓ Only index pages are read. The table (heap) is not accessed.

Performance Benefits



Fewer I/O Operations
No table (heap)
page reads



Lower CPU Usage
Less work to retrieve
rows



Better Cache Utilization
Smaller index fits in
memory



Faster Queries
Ideal for read-mostly
and analytical queries



Index Only Scan is the most efficient scan type when your index covers all required columns and visibility map is set.

Bitmap Heap Scan



Bitmap Heap Scan combines multiple index scans to find matching rows and then fetches the actual rows from the table in a sorted (by page) order.

Key Points

- Used when a query can use multiple indexes.
- Each index is scanned to build a bitmap of matching row locations (TIDs).
- Bitmaps are combined (AND / OR) to get final set of row locations.
- Heap pages are fetched in page order → fewer random I/Os.
- More efficient than multiple Index Scans or Sequential Scan for complex conditions.



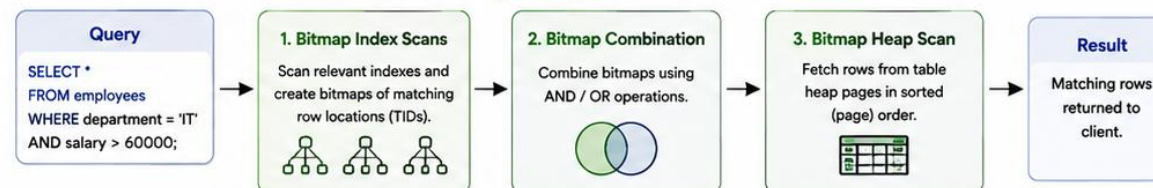
When is it OK?

When a query uses multiple indexes (AND / OR conditions) and matches a moderate number of rows spread across the table.

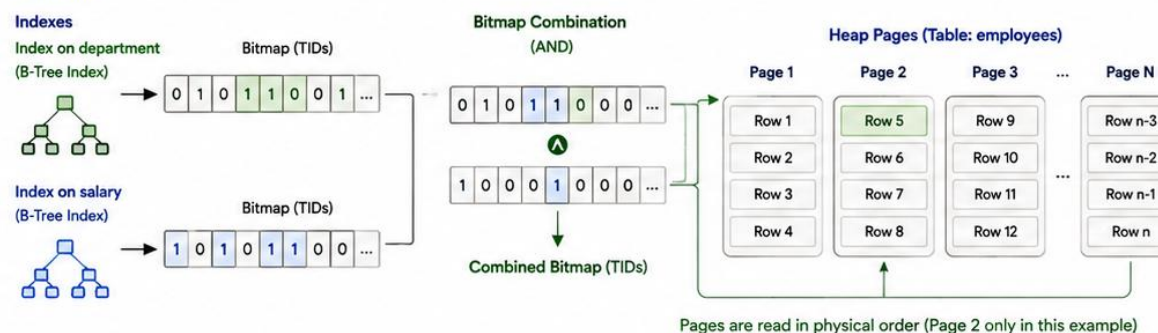


Goal: Reduce random I/O by combining index information and accessing table pages in an efficient order.

How a Bitmap Heap Scan Works



Example



Performance Benefits



Fewer random reads
Heap pages are read in sorted order.



Efficient for complex queries using multiple indexes.



Lower CPU usage
Bitmap operations are fast and memory based.



Better performance than Sequential Scan for moderate result sets.



Bitmap Heap Scan is ideal for queries with multiple index conditions that return a moderate number of rows.

TRY IT YOURSELF — EXERCISE 3: READ AN EXECUTION PLAN

Reinforces: the query planner, EXPLAIN, EXPLAIN ANALYZE, and the scan-type slides you just walked.

STEPS (notes/lab38-read-plan.md)

Step	What To Do
1 — EXPLAIN vs ANALYZE	Say what EXPLAIN ANALYZE gives you that EXPLAIN alone does not.
2 — Scan Types	Say what a sequential scan and an index scan each tell you about the query.
3 — Estimate Gap	Say what a large gap between estimated and actual rows means.
4 — Read Order	Say which end of the plan you read first, and why.

Plan reading

```
EXPLAIN          -> the plan and estimates, query not run
EXPLAIN ANALYZE -> actual rows and timing, query IS run
read innermost/most-indented first; big estimate gap -> run ANALYZE
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-read-the-plan.md (workspace: examples/module-38-exercises).

JOIN ALGORITHMS: NESTED LOOP, HASH, MERGE

Three join node types cover how PostgreSQL physically combines two row sets -- the planner chooses based on size and sort order.



Nested Loop Join

For each outer row, searches the inner side for matches -- efficient when the outer side is small or the inner side is indexed.



Hash Join

Builds a hash table from one side, then probes it with the other -- efficient for larger, unsorted inputs.



Merge Join

Requires both inputs sorted on the join column, then merges them in one pass -- efficient when both are already sorted, e.g. via an index.

KEY TAKEAWAY Nested Loop favors a small outer side or an indexed inner side; Hash Join favors larger unsorted inputs; Merge Join favors inputs that are already sorted on the join key.

Nested Loop Join



A **Nested Loop Join** compares each row from the outer table with rows in the inner table to find matching rows.

Key Points

- The outer table drives the join.
- For each row in the outer table, the inner table is scanned (or an index lookup is performed).
- Can use an index on the inner table for efficient lookups.
- Efficient for small outer tables and well-indexed inner tables.
- May be slow for large outer tables without indexes.



When is it Used?

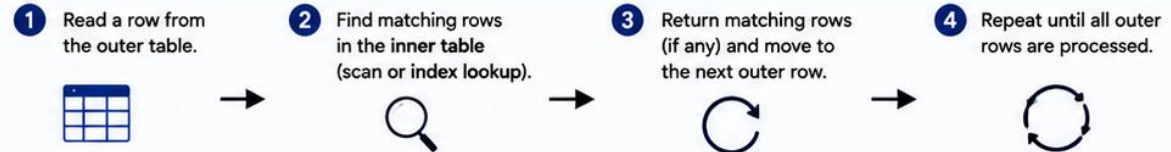
- ✓ Small result set from outer table.
- ✓ Inner table has an index on the join column.
- ✓ Large result set is not expected.



Goal

Find matching rows by looping through the outer table and searching the inner table for each row.

How Nested Loop Join Works



Example

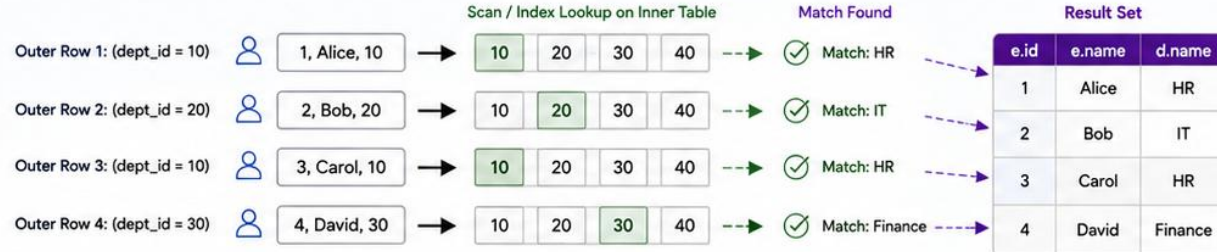
Query:
`SELECT *
FROM employees e
JOIN departments d
ON e.dept_id = d.id;`

Outer Table (employees)		
id	name	dept_id
1	Alice	10
2	Bob	20
3	Carol	10
4	David	30

Inner Table (departments)	
id	name
10	HR
20	IT
30	Finance
40	Sales

Join Condition

`e.dept_id = d.id`



Performance Characteristics



Good for small outer tables.



Very efficient when inner table has an index on join column.



Can be very slow if both tables are large and no indexes exist.



Complexity is $O(\text{outer_rows} \times \text{inner_lookup})$

Note



The optimizer chooses Nested Loop Join when it estimates it to be the lowest cost plan for the query.

Hash Join



A **Hash Join** builds a hash table on the smaller input (inner table) and then probes it with each row from the larger input (outer table) to find matches.

Key Points

- Build a hash table on the inner table using the join key.
- Scan the outer table and compute the hash for each row.
- Probe the hash table to find matching rows.
- Very efficient for large tables when no suitable indexes exist.
- Performance depends on memory for the hash table.



When is it Used?

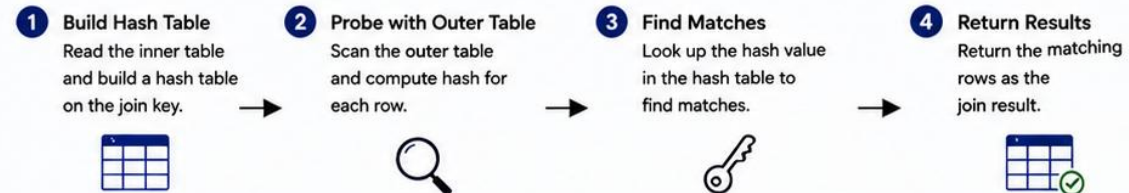
- ✓ Large tables with no useful indexes on join columns.
- ✓ Join returns a large number of rows.
- ✓ Inner table fits in memory to build the hash table.



Goal

Reduce I/O by avoiding repeated lookups.
Read each table once and use a hash table in memory to find matching rows quickly.

How Hash Join Works



Example

Query:
`SELECT *
FROM employees e
JOIN departments d
ON e.dept_id = d.id;`

Inner Table (departments) — Build Hash Table

id	name
10	HR
20	IT
30	Finance
40	Sales

Hash on id

Hash Table (in memory)

Hash Value	Rows (id → name)
h(10)	10 → HR
h(20)	20 → IT
h(30)	30 → Finance
h(40)	40 → Sales

Join Condition

`e.dept_id = d.id`

Outer Table (employees) — Probe Hash Table

id	name	dept_id
1	Alice	10
2	Bob	20
3	Carol	10
4	David	30
5	Eve	50

Compute hash(dept_id) and probe the hash table

Probe Lookup

h(10)
h(20)
h(10)
h(30)
h(50)

Match Found?

✓
✓
✓
✓
✗

Result Set

e.id	e.name	d.name
1	Alice	HR
2	Bob	IT
3	Carol	HR
4	David	Finance

Performance Characteristics



Excellent for large inputs without useful indexes.



Reads each table once (sequential scan).



Requires memory for the hash table (inner input).



Typically faster than Nested Loop Join for large data sets.



Note

The optimizer chooses Hash Join when it estimates it to be the lowest cost plan for the query.

Merge Join



A **Merge Join** efficiently joins two sorted inputs by scanning them in order and matching rows with equal join keys.

Key Points

- Both input tables must be sorted on the join key.
- Rows are compared in order, similar to the merge step in merge sort.
- When keys match, corresponding rows are joined.
- When keys don't match, the row with the smaller key is advanced.
- Efficient for large datasets and range-based joins.
- Can use indexes or explicit sort operations.



When is it Used?

- ✓ When both tables are already sorted on the join key.
- ✓ When indexes provide the required ordering.
- ✓ For range-based joins and large result sets.
- ✓ When Hash Join is not possible due to memory limits.



Goal

Join two sorted inputs by scanning them once each and producing the result in sorted order with minimal I/O.

How Merge Join Works



Example

Query:
`SELECT *
FROM employees e
JOIN departments d
ON e.dept_id = d.id;`

Employees (sorted by dept_id)

id	name	dept_id
1	Alice	10
3	Carol	10
2	Bob	20
4	David	30
5	Eve	50

Departments (sorted by id)

id	name
10	HR
20	IT
30	Finance
40	Sales

Merge Process

Compare Keys	Action
e.dept_id = 10 = d.id = 10	Match → output rows (Alice, HR) and (Carol, HR)
e.dept_id = 20 = d.id = 20	Match → output row (Bob, IT)
e.dept_id = 30 = d.id = 30	Match → output row (David, Finance)
e.dept_id = 50 > d.id = 40	Advance department pointer (no match)
e.dept_id = 50 > -end	Advance employee pointer (no match)

Pointer Movement Illustration



Performance Characteristics



Very efficient for large sorted inputs.



Performs well with range-based joins.



Output is produced in sorted order.



Low memory usage (compared to Hash Join).



Requires sorting if inputs are not already ordered.

Note



Merge Join is chosen when the optimizer estimates it will be cheaper than Hash Join or Nested Loop Join.

IDENTIFYING QUERY BOTTLENECKS

Finding a slow query in production requires the right tools -- guessing doesn't scale past a handful of queries.

Tool	What It Does
pg_stat_statements	Tracks every query's total time, mean time, and call count -- the single most important tool here.
log_min_duration_statement	Logs any query exceeding a configured duration, in milliseconds.
pg_stat_activity	Shows currently running queries and how long each has been running.
EXPLAIN ANALYZE	Deep-dives one specific query once it's been identified as slow.

KEY TAKEAWAY pg_stat_statements finds the slow queries; EXPLAIN ANALYZE explains why each one is slow -- the two tools work as a pair.

Identifying Query Bottlenecks



A **query bottleneck** is any part of a SQL statement that consumes excessive time or resources (CPU, I/O, memory), causing slow performance.

Common Bottlenecks



Full Table Scans

Scanning large tables when an index could be used.



Missing or Inefficient Indexes

Queries filtering or joining on columns without supporting indexes.



Expensive Joins

Inefficient join types or large intermediate result sets.



Sorts and Hashes

Large sorts or hash operations consume CPU and memory.



Functions on Columns

Using functions on indexed columns prevents index usage.



Excessive Data Processing

Retrieving more rows/columns than necessary (SELECT * or unfiltered data).



Key Takeaway

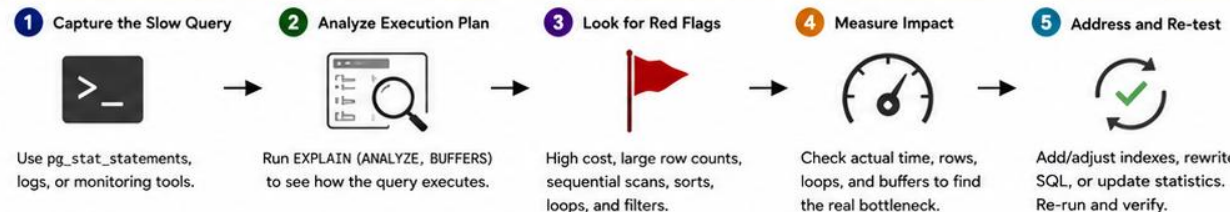
Focus on reducing the amount of data read, processed, and sorted to improve performance.



Goal

Find the operations that cost the most time and resources, then optimize to reduce I/O, CPU, and memory usage.

How to Identify Bottlenecks



Example: Analyzing an Execution Plan

```
SELECT e.name, d.name
FROM employees e
JOIN departments d ON e.dept_id = d.id
WHERE e.salary > 100000
ORDER BY e.hire_date;
```

Command Used:

```
EXPLAIN (ANALYZE, BUFFERS)
<query>;
```

Node	Actual Time (ms)	Rows	Loops	Buffers	Extra Info
Sort	512.345	15,432	1	8,203	Sort Key: e.hire_date
Hash Join	498.210	15,432	1	7,954	Hash Cond: e.dept_id = d.id
Seq Scan on employees e	320.112	250,000	1	5,120	Filter: (salary > 100000)
Hash	12.305	100	1	234	

Bottlenecks in This Plan

- Seq Scan on employees e – scanning 250,000 rows. ⚠
- Large Sort – consuming 512 ms and 8,203 buffer hits. ⚠
- High rows removed by filter (salary > 100000) – done after scan. ⚠

How to Fix

- ✓ Add index on employees(salary) or (salary, hire_date).
- ✓ Consider index on employees(dept_id) for join.
- ✓ Review query and filter earlier if possible.
- ✓ Re-run EXPLAIN ANALYZE to verify improvement.

Tools and Views



EXPLAIN (ANALYZE)

Shows actual execution plan with timing and row counts.



pg_stat_statements

Find top slow queries by mean time and calls.



auto_explain

Automatically log slow or complex queries.



pg_stat_activity

Monitor currently running queries.



Buffers

Check shared/local buffer hits and reads.

Red Flags to Watch For



Seq Scan on Large Tables



High Actual Time vs Planning Time



Large Rows Removed by Filter



Many Loops on Expensive Nodes



High Buffers Read vs Hit



Disk I/O (Reads)



Remember

Always measure, don't guess. The execution plan tells the true story.

AVOIDING SELECT *

*SELECT * is convenient to write and costly to run at scale -- it transfers unused data and can block an index-only scan.*

```
-- Costly: transfers every column, blocks an index-only scan
SELECT * FROM customer WHERE status = 'ACTIVE';

-- Better: only the columns the caller actually uses
SELECT customer_id, full_name, status FROM customer WHERE status = 'ACTIVE';
```

More network bytes transferred, whether or not the caller uses every column.
Prevents an Index Only Scan the moment even one selected column isn't in the index.
Silently breaks assumptions if the table's columns change later.

KEY TAKEAWAY Selecting only the columns actually needed reduces network transfer and, when every requested column happens to be indexed, can unlock a dramatically faster Index Only Scan.

Avoiding `SELECT \*`



Best Practice: Always specify the columns you need instead of using `SELECT \*`. This improves performance, reduces network usage, and makes your SQL intention clear.



Bad Practice

Retrieves all columns even if you only need a few.
Leads to unnecessary data transfer and processing.

```
SELECT *  
FROM employee  
WHERE department_id = 10;
```

- ⚠ Transfers more data than needed
- ⚠ Slower performance, especially on large tables
- ⚠ Breaks when new columns are added
- ⚠ Exposes sensitive or unnecessary data
- ⚠ Makes queries harder to read and maintain



Good Practice

Retrieves only the required columns.
Improves performance and clarity.

```
SELECT id, first_name, last_name, email  
FROM employee  
WHERE department_id = 10;
```

- ✓ Transfers only the data you need
- ✓ Faster queries and better performance
- ✓ Stable even if table structure changes
- ✓ Reduces risk of exposing unnecessary data
- ✓ Clear, maintainable, and self-documenting SQL



Key Takeaway: Be intentional. Select only what you use.
Efficient SQL leads to faster applications and happier databases.



FUNCTIONS ON INDEXED COLUMNS

Wrapping an indexed column in a function usually prevents PostgreSQL from using a plain index on that column at all.

```
-- Non-sargable: index on email cannot be used
SELECT * FROM customer WHERE LOWER(email) = 'amina@example.com';

-- Sargable: index on email is used directly
SELECT * FROM customer WHERE email = 'amina@example.com';
```

LOWER(email) computes a new value for every row before comparing it, so the plain index on email can't be used -- a sequential scan follows. Rewriting the literal instead of the column keeps the comparison sargable -- 'amina@example.com' is already lowercase, so no function is needed. When the function truly must stay in WHERE, an expression index -- CREATE INDEX ON customer (LOWER(email)) -- restores index use.

KEY TAKEAWAY A condition is 'sargable' -- search-argument-able -- when it's written in a form the optimizer can match directly against an index; wrapping a column in a function usually isn't.

Functions on Indexed Columns



Applying a function to an indexed column often prevents the database from using the index, leading to full table scans and poor performance.

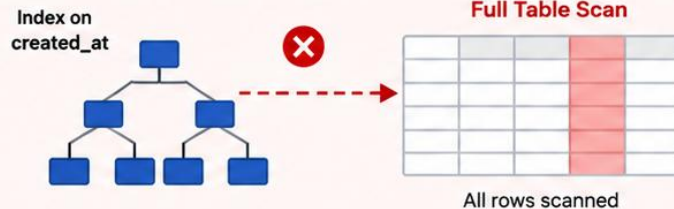
✗ Bad Practice

Function applied to the indexed column (created_at).
Index cannot be used → Full table scan.

```
SELECT *  
FROM orders  
WHERE DATE(created_at) = '2024-05-01';
```

What happens?

- Database must compute the function for every row.
- Index on created_at cannot be used.
- Results in a full table scan.
- Slower queries, especially on large tables.



✓ Good Practice

Use a range condition on the indexed column.
Index can be used → Fast query.

```
SELECT *  
FROM orders  
WHERE created_at >= '2024-05-01'  
AND created_at < '2024-05-02';
```

Why this is better?

- No function on the column.
- Index on created_at can be used.
- Database directly navigates to matching rows.
- Faster and more scalable.



Key Takeaway: Avoid applying functions to indexed columns in WHERE clauses. Instead, rewrite queries to use range conditions so the database can use indexes efficiently.

Common functions to avoid on indexed columns:

UPPER() LOWER() DATE() TO_CHAR() SUBSTRING() COALESCE() NVL()

N+1 QUERY PROBLEM

The N+1 problem: one query to fetch N parent rows, then one additional query per row to fetch related data -- N+1 round trips instead of one or two.

```
-- 1 query for customers, then N queries -- one per customer -- for their accounts
SELECT * FROM customer WHERE status = 'ACTIVE';           -- 1 query, N rows
SELECT * FROM account WHERE customer_id = ?;              -- run N times

-- Fixed: one query total
SELECT * FROM customer c JOIN account a ON a.customer_id = c.customer_id
WHERE c.status = 'ACTIVE';
```

One query returns N customer rows; the application loop then fires one more query per customer for its accounts -- N+1 round trips total. Each round trip pays full network and parsing overhead, so N+1 gets dramatically slower as the result set grows, even though each query is individually fast.

A single JOIN, or a batched WHERE customer_id IN (...), replaces all N+1 calls with one round trip and one execution plan.

KEY TAKEAWAY N+1 usually isn't visible in the SQL itself -- it shows up as an application loop issuing one query per row, and it becomes the module 39 discussion once JPA generates the loop automatically.

N+1 Query Problem

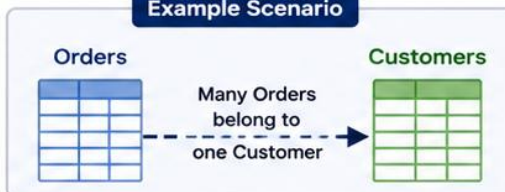


The **N+1 query problem** occurs when an application executes **1 query** to load a set of records (**N**) and then executes **1 additional query** for each record to load related data.

❌ The Problem

- 1 query to fetch all parents (e.g., Orders)
- **N** additional queries to fetch related children (e.g., Customers) one by one.
- Leads to **many database round-trips, high latency, and poor performance.**

Example Scenario



✅ The Solution

- Fetch all required data using a **JOIN** or **FETCH JOIN**.
- Retrieve related data in a single query.
- **Reduces database load and improves performance.**

N+1 Query Example (Inefficient)

Step 1: Fetch all orders

```
SELECT *  
FROM orders;
```

order_id	customer_id
1	101
2	102
3	103
4	104
5	105
...	...

Step 2: For each order, fetch customer
(N additional queries)

```
SELECT * FROM customers WHERE id = 101;
```

```
SELECT * FROM customers WHERE id = 102;
```

```
SELECT * FROM customers WHERE id = 103;
```

```
SELECT * FROM customers WHERE id = 104;
```

```
SELECT * FROM customers WHERE id = 105;
```

...



Total Queries: N + 1

More round-trips = Slower performance

Optimized Solution (Efficient)

Single Query using JOIN

```
SELECT o.id AS order_id,  
       o.customer_id,  
       c.name AS customer_name  
FROM orders o  
JOIN customers c ON c.id = o.customer_id;
```

order_id	customer_id	customer_name
1	101	Alice
2	102	Bob
3	103	Charlie
4	104	David
5	105	Eva
...	...	...



Benefits

- Only 1 query
- Fewer round-trips
- Lower database load
- Faster response time
- Better scalability



Total Queries: 1

Fewer round-trips = Better performance



How to Prevent N+1

- Use JOINS or FETCH JOINS in queries.
- Use @EntityGraph in Spring Data JPA.
- Use batch fetching or subselect fetching when appropriate.



Best Practices

- Always analyze query patterns and execution plans.
- Monitor SQL logs during development and testing.
- Fetch only what you need, but fetch it in the most efficient way.



OVER-INDEXING

More indexes are not automatically better -- every index adds write overhead and storage, whether or not it's actually used.



Slower Writes

Every INSERT, UPDATE, or DELETE must maintain every index on that table.



Wasted Storage

An unused or rarely-used index still consumes real disk space.



Planner Confusion

More index choices can occasionally lead the planner to a worse plan than fewer, clearer options.



The Fix

Index based on real, measured query patterns -- verify each one earns its cost with EXPLAIN, not guesswork.

KEY TAKEAWAY Add an index only when EXPLAIN shows a real query pattern needs it -- every unused index is pure write-overhead cost with zero benefit.

Over-Indexing



Over-indexing occurs when a table has more indexes than necessary.

Too many indexes can slow down write operations and consume storage without significant read benefits.

! The Problem

- Indexes improve read performance, but every index has a cost.
- Too many indexes create overhead for INSERT, UPDATE, and DELETE.
- Storage, memory, and maintenance time increase.
- Can lead to slower overall performance.



🔍 Signs You May Be Over-Indexing

- Many indexes are rarely or never used.
- Write operations are slower than expected.
- High storage usage for index pages.
- Index maintenance tasks take too long.
- Execution plans do not use most of the indexes.

Check Unused Indexes

```
SELECT schemaname, tablename, indexname,  
       idx_scan, idx_tup_read  
FROM   pg_stat_user_indexes  
ORDER BY idx_scan ASC;
```

→ Indexes with idx_scan = 0
may be unnecessary.

📊 The Impact



Slower Writes

Every INSERT, UPDATE, and DELETE must update every index.



More Storage

Each index consumes disk space.



Higher Maintenance

More time spent rebuilding, reindexing, and maintaining indexes.



Increased Memory Usage

Indexes are cached in memory, reducing space for other data.



Slower Overall Performance

Write-heavy systems can see significant slowdowns.



Best Practices

- ✓ Create indexes based on real query patterns.
- ✓ Avoid indexing low-cardinality columns.
- ✓ Use composite indexes selectively.
- ✓ Regularly review and remove unused indexes.
- ✓ Monitor index usage with execution plans and statistics.
- ✓ Rebuild indexes when fragmentation becomes high.



Key Takeaway: Indexes are powerful, but more is not better.

Create only the indexes you need, monitor their usage, and remove the ones that do not provide value.



TRY IT YOURSELF — EXERCISE 4: FIX SARGABILITY AND N+1

*Reinforces: query bottlenecks, SELECT *, functions on indexed columns, the N+1 problem, and over-indexing as you just covered.*

STEPS (notes/lab38-antipatterns.md)

Step	What To Do
1 — Sargable Rewrite	Rewrite a predicate that wraps an indexed column in a function.
2 — N+1 Shape	Describe the N+1 pattern in the CRM and the query count it produces.
3 — N+1 Fix	Give the fix that replaces the loop, and say why a cache is not it.
4 — SELECT * Cost	Say what SELECT * costs beyond extra bytes.

Antipattern fixes

```
bad  where date(created_at) = '2026-01-01'
good where created_at >= '2026-01-01' and created_at < '2026-01-02'
N+1: 1 query for customers + N for accounts -> one join or one IN query
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-sargable-and-nplus1.md (workspace: examples/module-38-exercises).

STATISTICS AND ANALYZE

The query planner's cost estimates come from statistics about each table -- ANALYZE is what keeps them fresh.

```
ANALYZE customer;    -- refresh statistics for one table
ANALYZE;              -- refresh statistics for every table
```

Statistics include row counts, most-common values, and how selective each column typically is. PostgreSQL's autovacuum daemon runs ANALYZE automatically in most configurations. Stale statistics, after a large bulk load, are a common cause of a suddenly bad plan.

KEY TAKEAWAY Stale statistics are one of the most common causes of a sudden, unexplained bad plan -- running ANALYZE after a large data change is often the entire fix.

Statistics and ANALYZE



PostgreSQL uses table and index statistics to help the query planner choose the most efficient execution plan.
ANALYZE collects or refreshes these statistics.



Why Statistics Matter

- The query planner is cost-based and relies on statistics.
- Accurate statistics = better cardinality estimates.
- Better estimates lead to better plan choices (index vs. seq scan, join type, join order).
- Outdated statistics can cause the planner to choose suboptimal plans.

How It Works



PostgreSQL stores statistics (row counts, distinct values, null fractions, most common values, histograms, etc.).



The planner uses these statistics to estimate the cost of different execution plans.



The plan with the lowest estimated cost is chosen.



Problem: Outdated Statistics

Data Changes

id	status
1	NEW
2	NEW
3	NEW
...	...
1000	NEW



After Many Changes

id	status
1	SHIPPED
2	SHIPPED
3	CANCELLED
...	...
1000	DELIVERED

- ✗ Planner thinks most rows are "NEW".
- ✗ Chooses index scan.
- ✗ But most rows are "SHIPPED" or "DELIVERED".
- ✗ Results in slow queries (sequential scans or bad join choices).



Solution: Use ANALYZE

Run ANALYZE

```
ANALYZE;  
-- or  
ANALYZE orders;
```

Benefits

- ✓ Refreshes table and index statistics.
- ✓ Improves cardinality estimates.
- ✓ Enables the planner to choose optimal execution plans.
- ✓ Better performance and scalability.



When to Run ANALYZE

- After large data changes (bulk INSERT/UPDATE/DELETE).
- After nightly batch jobs or ETL processes.
- When query performance suddenly degrades.
- Rely on autovacuum/analyze, but consider manual ANALYZE for critical tables.



Check Statistics

```
SELECT relname, n_live_tup, n_dead_tup, last_analyze  
FROM pg_stat_user_tables  
ORDER BY last_analyze NULLS FIRST;
```

`n_live_tup` – estimated live rows
`n_dead_tup` – estimated dead rows
`last_analyze` – last analyze time



Key Takeaway: Fresh statistics are essential for the PostgreSQL optimizer to make the right decisions.
Use **ANALYZE** regularly to keep statistics up to date and your queries fast.

VACUUM OVERVIEW

PostgreSQL's MVCC model leaves behind dead row versions -- VACUUM reclaims that space and keeps tables healthy.

Every UPDATE and DELETE leaves the old row version behind, marked dead, not immediately erased -- this is how MVCC gives concurrent readers a consistent view.

VACUUM scans a table and reclaims the space dead row versions occupy, making it available for future INSERTs and UPDATEs.

autovacuum runs VACUUM (and ANALYZE) automatically in the background for essentially every table in normal operation.

A table that's never vacuumed enough accumulates 'bloat' -- wasted space that slows down every scan of that table.

KEY TAKEAWAY VACUUM is MVCC's cleanup crew -- it reclaims space left behind by updates and deletes, and autovacuum runs it automatically so students rarely need to invoke it by hand.

VACUUM Overview



VACUUM reclaims storage occupied by dead tuples and keeps the database running efficiently. It also updates statistics that help the query planner make better decisions.



Why VACUUM Matters

- ✓ PostgreSQL uses MVCC (Multi-Version Concurrency Control).
- ✓ UPDATE and DELETE do not remove rows immediately.
- ✓ Old row versions (dead tuples) remain until vacuumed.
- ✓ Dead tuples consume disk space and slow down queries.
- ✓ VACUUM reclaims space and keeps performance stable.



What VACUUM Does

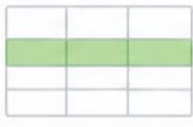
- ✓ Removes dead tuples – Frees space used by old row versions.
- ✓ Marks pages as reusable – Space becomes available for new data.
- ✓ Updates visibility map – Helps index-only scans work efficiently.
- ✓ Updates statistics (with ANALYZE) – Improves planner decisions.
- ✓ Prevents table and index bloat – Keeps performance predictable.



How Dead Tuples Accumulate

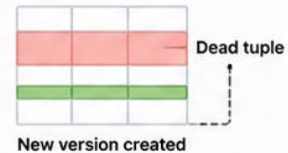
1. Insert

Row inserted



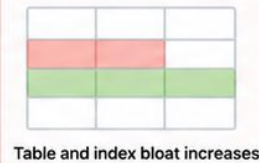
2. Update / Delete

Old version becomes dead



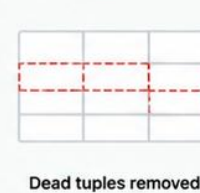
3. Before VACUUM

Dead tuples still occupy space



VACUUM Reclaims Space

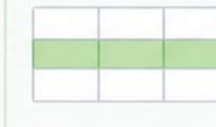
4. VACUUM Runs



Dead tuples removed

5. After VACUUM

Space is free and reusable



Benefits

- Faster queries
- Smaller table and index size
- Better cache usage
- More predictable performance
- Healthier long-running systems



Types of VACUUM

- VACUUM – Reclaims space.
- ANALYZE – Updates statistics.
- VACUUM ANALYZE – Does both.
- VACUUM FULL – Rewrites the table to recover space to the OS (takes exclusive lock; avoid on busy systems).

```
VACUUM table_name;  
ANALYZE table_name;  
VACUUM ANALYZE table_name;  
VACUUM FULL table_name;
```



When to Run VACUUM

- After large INSERT/UPDATE/DELETE activity.
- When table bloat is observed.
- When autovacuum cannot keep up.
- For critical tables, monitor and vacuum proactively.

Related Views (Monitor)

```
pg_stat_user_tables  
pg_stat_all_tables  
pg_stat_progress_vacuum
```



Key Takeaway: VACUUM is essential for PostgreSQL health. Regular vacuuming prevents bloat, reclaims space, and keeps your database fast and efficient.

Autovacuum

PostgreSQL runs autovacuum automatically, but tuning may be needed for high-throughput systems.



PAGINATION PERFORMANCE

Returning a huge result set at once is rarely necessary -- pagination limits each request to a manageable page of rows.

```
SELECT * FROM customer ORDER BY customer_id LIMIT 20 OFFSET 40;  -- page 3, 20 rows per page
```

LIMIT caps how many rows come back; OFFSET skips a number of rows before starting.

Always pair LIMIT with an explicit ORDER BY -- without one, row order, and therefore which page you get, is not guaranteed.

OFFSET's cost grows with page depth -- the next slide covers why, and the alternative.

KEY TAKEAWAY LIMIT and OFFSET are the basic pagination tools -- always pair them with an explicit ORDER BY, since PostgreSQL makes no promise about row order otherwise.

Pagination Performance



Poor pagination strategies can slow down queries as data volume grows.
Understanding how pagination works helps you choose the most efficient approach.



Why Pagination Performance Matters

- Large OFFSET values force the database to scan and discard rows.
- Response time increases as users move to later pages.
- High latency and resource usage impact user experience.
- Use the right pagination strategy for scale.



Key Considerations

- Data volume and growth rate
- Access pattern (sequential vs random page access)
- Consistent sorting (unique and stable order)
- Index support for sort columns
- Correct pagination strategy for the use case

Offset-Based Pagination (LIMIT ... OFFSET ...)

```
SELECT id, name, created_at
FROM orders
ORDER BY created_at DESC
LIMIT 20 OFFSET 100000;
```

- ❌ Database must read and skip 100,000 rows before returning 20.
- ❌ Work increases as page number grows.
- ❌ Slower response time for deep pages.
- ❌ Not suitable for large datasets.



Performance Impact: Gets progressively slower as OFFSET increases.

Keyset (Cursor)-Based Pagination

```
SELECT id, name, created_at
FROM orders
WHERE (created_at, id) < ('2024-05-10 10:00:00', 987654)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

- ✅ Uses the last seen key (cursor).
- ✅ Database can use index to find next set.
- ✅ Consistent performance for deep pages.
- ✅ Best for large and fast-growing datasets.



Performance Impact: Remains fast and predictable even for very deep pages.



When to Use What?

Strategy	Use When
Offset-Based	• Small to medium datasets • Users need random access to pages (e.g., go to page 10) • Administrative back-office tools
Keyset-Based	• Large datasets • Sequential navigation (next/previous) • Real-time feeds, activity streams, infinite scroll



Best Practices

- Always use a stable and unique sort order.
- Prefer keyset pagination for large datasets.
- Avoid SELECT *; select only required columns.
- Ensure proper indexing on sort columns.
- Do not expose deep page numbers in UI.

Index Example

```
-- For ORDER BY created_at DESC, id DESC
CREATE INDEX idx_orders_created_at_id_desc
ON orders (created_at DESC, id DESC);
```

Helps both offset and keyset pagination.



Key Takeaway: Offset-based pagination slows down as page number increases.
For high performance at scale, use keyset (cursor)-based pagination with proper indexing.

Page Navigation Example

Offset-Based: 1 3 ... 100 101 102 ... 10000
Keyset-Based: [First] [Prev] [Next] [Last]

OFFSET VS KEYSET PAGINATION

Keyset pagination filters on the last-seen row's key instead of skipping a count -- its cost stays flat no matter how deep the page.

```
-- Offset: cost grows with page depth
SELECT * FROM customer ORDER BY customer_id LIMIT 20 OFFSET 100000;

-- Keyset: cost stays flat -- filters directly using an index
SELECT * FROM customer WHERE customer_id > :last_seen_id
ORDER BY customer_id LIMIT 20;
```

OFFSET 100000 still makes PostgreSQL scan and discard the first 100,000 matching rows before it can return page 5001.
Keyset pagination filters directly on the last-seen key, so an index seek jumps straight to the next page regardless of depth.
Keyset requires a stable, unique ORDER BY (or a tie-breaker column) so no row is ever skipped or repeated between page requests.

KEY TAKEAWAY Keyset pagination replaces 'skip N rows' with 'give me rows after this specific key' -- an index on the ordering column makes every page equally cheap, however deep.

Offset vs Keyset **Pagination**



Two common pagination strategies in PostgreSQL. Choosing the right one can dramatically **impact performance**, especially for **large and fast-growing datasets**.

Offset-Based Pagination (LIMIT ... OFFSET ...)

- Database must scan and skip rows up to the OFFSET.
- Work increases as page number grows.
- Slower for deep pagination on large tables.

How It Works

- Scan from the start.
- Skip first 100,000 rows.
- Return the next 20 rows.

```
SELECT id, name, created_at
FROM orders
ORDER BY created_at DESC
LIMIT 20 OFFSET 100000; -- Page 5001 (20 rows per page)
```

Example
Row Scan Order: 1 2 3 4 5 ... 99998 99999 100000 100001 100002 ...
100,000 rows scanned and discarded | 20 rows returned

Performance Impact: Gets slower as OFFSET increases.

Keyset (Cursor)-Based Pagination

- Uses the last seen key (cursor) to fetch the next set.
- Uses index to jump directly to the next set.
- Consistent performance even for deep pages.
- Resilient to data changes between requests.

How It Works

- Use last seen (created_at, id).
- Database uses index to find next rows.
- Return the next 20 rows.

```
SELECT id, name, created_at
FROM orders
WHERE (created_at, id) < ('2024-05-10 10:00:00', 987654)
ORDER BY created_at DESC, id DESC
LIMIT 20; -- Next page after the cursor
```

Index Scan (uses index) ... 987652 987653 987654 987655 987656 987657 ...
Last seen (created_at, id) | 20 rows returned

Performance Impact: Remains fast and predictable even for very deep pages.

Aspect	Offset-Based Pagination	Keyset (Cursor)-Based Pagination	Winner
Performance (Deep Pages)	Slow – scans and skips many rows	Fast – uses index to jump	✓ Keyset
Scalability	Poor – degrades as table grows	Excellent – consistent performance	✓ Keyset
Consistency with Data Changes	Prone to duplicates or missing rows	More stable with changes	✓ Keyset
Implementation Complexity	Simple	Moderate (requires cursor handling)	✓ Offset
Random Page Access (e.g., go to page 100)	Easy	Not practical	✓ Offset

When to Use What?

Use Offset-Based When:

- Small to medium datasets
- Users need random access to pages (e.g., go to page 10)
- Administrative back-office tools
- Simple reporting screens

Use Keyset-Based When:

- Large datasets
- Infinite scroll or next/previous navigation
- Real-time feeds, activity streams
- High performance is critical

Best Practices

Always have a stable and unique sort order (e.g., created_at, id).

Index the columns used for sorting and filtering.

Avoid exposing deep page numbers in the UI.

Prefer keyset pagination for APIs and infinite scroll.

Test both strategies and choose based on data size and access patterns.

TRY IT YOURSELF — EXERCISE 5: COMPARE OFFSET AND KEYSET PAGING

Reinforces: statistics, VACUUM, pagination performance, and the offset-versus-keyset slides you just covered.

STEPS (notes/lab38-pagination.md)

Step	What To Do
1 — Offset Cost	Say what the database does to serve OFFSET 100000 LIMIT 20.
2 — Keyset Query	Write the keyset query that fetches the page after a known row.
3 — Trade-off	Say what keyset pagination gives up compared with offset.
4 — Stable Order	Say why the sort must be deterministic, and how to guarantee it.

Paging strategies

```
offset: scans and discards 100000 rows, then returns 20
keyset: where (opened_at, id) < (:lastOpenedAt, :lastId) order by ... limit 20
ties: always include a unique tiebreaker in the ORDER BY
```

TRY IT NOW — Complete Exercise 5 before continuing: `exercises/exercise-05-pagination-performance.md` (workspace: `examples/module-38-exercises`).

MEASURING QUERY PERFORMANCE

Tuning without measurement is guessing -- always capture a real before number before changing anything.

Metric	Where To Get It
Actual execution time	EXPLAIN ANALYZE's actual time value.
Total/mean time across calls	pg_stat_statements.
Rows examined vs. rows returned	EXPLAIN ANALYZE's actual rows at each node.
Index usage	pg_stat_user_indexes.idx_scan.

KEY TAKEAWAY Capture a real, measured baseline with EXPLAIN ANALYZE and pg_stat_statements before making any change, then measure again after -- a felt improvement isn't evidence.

16

Measuring Query Performance

You can't improve what you don't measure. PostgreSQL provides powerful tools to measure, analyze, and optimize query performance.



Key Performance Metrics



Execution Time

Total time taken to execute the query.



Rows Scanned vs Rows Returned

High scanned/returned ratio indicates inefficiency.



Planning Time

Time spent by the planner to create the execution plan.



CPU and I/O Usage

High CPU or disk I/O can indicate bottlenecks.



Buffers and Cache Hits

Shows how effectively data is cached.

Tools and Commands



EXPLAIN

Shows the execution plan without running the query.



EXPLAIN ANALYZE

Runs the query and shows the actual execution statistics.



pg_stat_statements

Tracks query statistics across the database to find slow queries.

EXAMPLE

```
-- Analyze a query
EXPLAIN ANALYZE SELECT e.id, e.name, d.name
FROM employee e
JOIN department d ON e.department_id = d.id
WHERE e.status = 'ACTIVE';
```



```
QUERY PLAN
(actual time=0.123..1.456 rows=250 loops=1)
-----
Nested Loop (cost=0.56..123.45 rows=250)
(actual time=0.123..1.456 rows=250)
-> Index Scan using idx_emp_status on employee e
(cost=0.28..45.67 rows=250)
(actual time=0.045..0.320 rows=250)
-> Index Scan using dept_pkey on department d
(cost=0.28..0.30 rows=1)
(actual time=0.003..0.003 rows=1)

Planning Time: 0.210 ms
Execution Time: 1.489 ms
```

Best Practices



Measure Before Optimizing

Identify actual bottlenecks before making changes.



Use Production-Like Data Volumes

Test with realistic data to get accurate results.



Compare Before and After

Measure improvements after each optimization.



Monitor Continuously

Performance can degrade over time as data and usage grow.



Use the Right Indexes

Validate that indexes are used and effective.



QUERY TUNING WORKFLOW

A repeatable five-step process turns query tuning from guesswork into a discipline that works on any slow query.

1 Identify

Find the slow query with `pg_stat_statements` or application monitoring -- never guess which query is the problem.

2 Measure & Analyze

Capture a baseline and run `EXPLAIN ANALYZE` to see the real plan and where time actually goes.

3 Apply a Fix

One targeted change at a time -
- an index, a rewritten condition,
a different join shape.

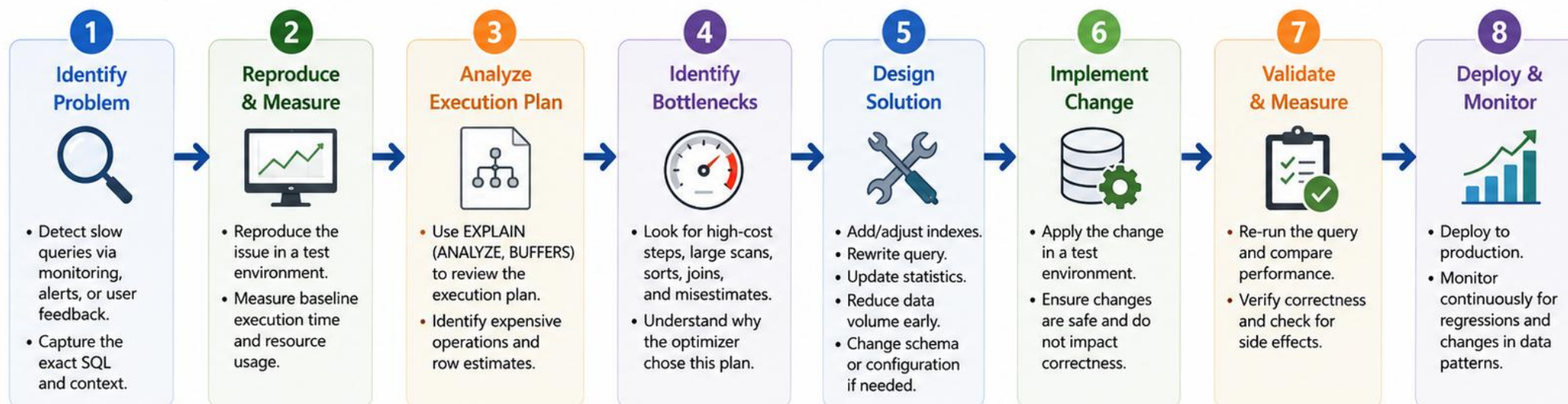
4 Verify & Repeat

Re-run `EXPLAIN ANALYZE` and compare against the baseline -- if not fixed, or a new bottleneck appears, repeat.

KEY TAKEAWAY Identify, measure, fix one thing, verify, repeat -- this exact loop is what Lab 38 grades, and it's the process for any slow query, not just the ones in this deck.

48 Query Tuning Workflow

A repeatable, data-driven process to **identify, analyze, and resolve** SQL performance issues.



Key Principles



Measure First
Always measure before making changes.



Small Changes
Iterate with small, reversible changes.



Data Matters
Performance depends on data volume and distribution.



Correctness First
Never sacrifice correctness for performance.



Document
Record what was changed and why, with results.



Monitor Continuously
Data and workloads change—keep observing.



Common Tools



EXPLAIN
View the estimated execution plan.



EXPLAIN ANALYZE
See actual execution time and row counts.



pg_stat_statements
Find slow and frequently executed queries.



pg_stat_activity
Monitor running queries and waits.



auto_explain
Log execution plans for slow queries.



Module 38: SQL and Query Performance with PostgreSQL

Measure. Analyze. Optimize. Repeat.



BEFORE-AND-AFTER QUERY OPTIMIZATION

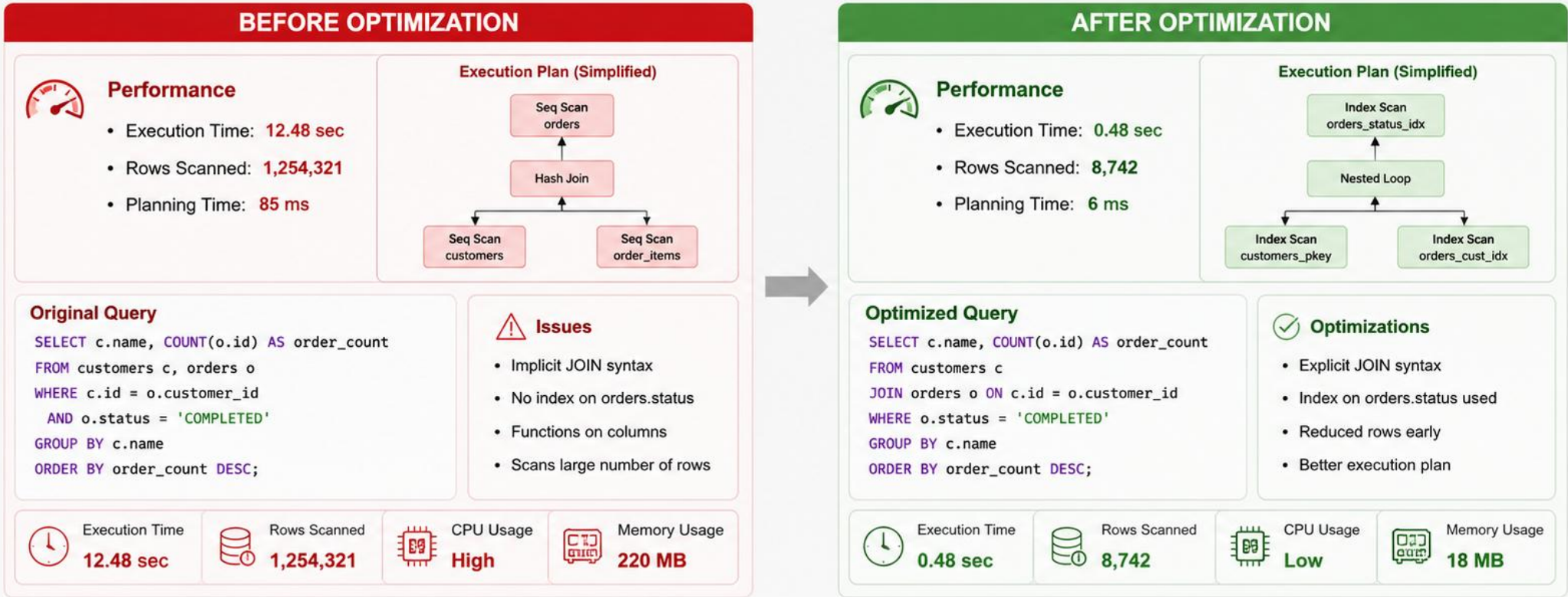
One real worked example, applying the tuning workflow end to end against a genuinely slow CRM query.

Stage	Query & Result
Before	SELECT * FROM account WHERE LOWER(status) = 'active'; -- Seq Scan, actual time 118ms, rows=4,812 examined for 812 matches.
Diagnosis	Non-sargable LOWER(status) defeats the index; SELECT * blocks an Index Only Scan.
After	SELECT account_id, customer_id, balance FROM account WHERE status = 'ACTIVE'; -- Index Only Scan, actual time 2.1ms.

KEY TAKEAWAY Same business question, 118ms down to 2.1ms -- purely from applying sargability and column-selection habits already covered this module, verified with EXPLAIN ANALYZE.

BEFORE-AND-AFTER QUERY OPTIMIZATION

Optimizing SQL queries improves performance by reducing execution time, resource usage, and system load.



ENTERPRISE SQL BEST PRACTICES

A consolidated checklist -- every practice here traces back to a specific slide earlier this module.

Practice	In Short
Write Sargable Queries	Avoid functions on indexed columns; use expression indexes when transformation is unavoidable.
Select Only What's Needed	Never SELECT * in application code -- reduce transfer, enable index-only scans.
Index With Intent	Selectivity and column order matter -- verify usage, remove unused indexes.
Read the Plan, Don't Guess	EXPLAIN ANALYZE before and after every tuning change.
Keep Statistics Fresh	Trust autovacuum; run ANALYZE manually after large bulk changes.
Paginate for Scale	Prefer keyset pagination for deep, growing result sets.

KEY TAKEAWAY Every row in this table is a direct callback to a slide already covered -- treat this as the pre-submission checklist for Lab 38's diagnostic report.

50 Enterprise SQL Best Practices

Write efficient, secure, maintainable, and scalable SQL that performs well in enterprise environments.



1 Understand Your Data



- Know your schema, data types, relationships.
- Understand data volumes and distribution.
- Use EXPLAIN to understand how queries are executed.

2 Write Set-Based SQL



- Use set-based operations instead of row-by-row processing.
- Avoid cursors and loops when set-based SQL can solve the problem.

3 Select Only What You Need



- Avoid SELECT *.
- Retrieve only the columns required.
- Reduces I/O, memory usage, and network traffic.

4 Use Proper Indexes



- Index columns used in WHERE, JOIN, ORDER BY, GROUP BY.
- Use composite indexes wisely.
- Regularly review and remove unused indexes.

5 Write Sargable Predicates



- Avoid functions on indexed columns.
- Use operators that allow index usage.
- Example: use `column >= :val` not `DATE(column) = :val`.

6 Avoid Unnecessary Sorting



- Avoid ORDER BY if not needed.
- Ensure sorting columns are indexed.
- Minimize sorts on large result sets.

7 Use Joins Effectively



- Use appropriate join types.
- Ensure join columns are indexed.
- Validate join conditions to avoid Cartesian products.

8 Filter Early, Aggregate Late



- Apply filters in WHERE clause to reduce rows early.
- Aggregate only after the data set is reduced.
- Improves performance significantly.

9 Handle Large Data Sets



- Use pagination (keyset pagination preferred).
- Process data in batches when needed.
- Avoid loading large data sets into memory.

10 Monitor and Continuously Improve



- Monitor slow queries and wait events.
- Keep statistics up to date (ANALYZE).
- Review and optimize queries regularly.



Additional Enterprise Guidelines



Ensure Security
Use bind variables, apply least privilege, and validate input.



Ensure Maintainability
Use consistent formatting, meaningful aliases, and clear naming.



Standardize SQL
Follow coding standards and templates across teams.



Consider Cost
Understand cost vs benefit for indexes, partitions, and changes.



Document Complex SQL
Explain intent, logic, assumptions, and any non-obvious decisions.



TRY IT YOURSELF — EXERCISE 6: PLAN THE TUNING EVIDENCE

Pre-lab gate: plan the measure-change-measure workflow and its evidence, then confirm the five earlier notes files are complete.

STEPS (notes/lab38-readiness.md)

Step	What To Do
1 — Workflow	Write the tuning loop in order, starting from measurement.
2 — Evidence Pair	Say what a before-and-after record must contain to be credible.
3 — One Change	Say why only one change is made between measurements.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

Tuning loop

```
measure (EXPLAIN ANALYZE, BUFFERS) -> change ONE thing -> measure again
record: query, plan shape, actual rows, timing, buffers -- both runs
keep the change only if the evidence says it helped
```

TRY IT NOW — Complete Exercise 6 before Lab 38: exercises/exercise-06-lab38-readiness.md (workspace: examples/module-38-exercises).

LAB OVERVIEW — SQL AND QUERY PERFORMANCE WITH POSTGRESQL

Tune CRM SQL with joins, CTEs, aggregations, EXPLAIN, indexes, pagination /

BUSINESS SCENARIO

- Before Spring Data JPA (Lab 39), leadership freezes:

LEARNING OBJECTIVES

- Read PostgreSQL `EXPLAIN` / `EXPLAIN ANALYZE` plans (seq vs index scans, joins) Build joins, CTEs, and aggregations for CRM list/report queries Choose indexes by selectivity and prove them with before/after plans Avoid `SELECT *` and recognize N+1 query patterns Implement keyset pagination as an alternative to deep `OFFSET`

WHAT YOU BUILD

- `examples/lab38-crm/` — performance SQL + report | Baseline + after-index plans · keyset demo · `report.md` |

KEY TAKEAWAY Evidence-based **PostgreSQL** tuning for Northstar CRM: generate volume, capture actual plans, create selective indexes, rewrite non-sargable predicates, compare joins/CTEs/aggregations, avoid `SELECT *`, contrast `OFFSET` vs keyset paging, and document results...



Lab Overview – SQL and Query Performance

In this lab, you will write, analyze, and optimize SQL queries in PostgreSQL to improve performance using indexing, execution plans, and best practices.



Lab Objectives

- ✓ Analyze query performance using EXPLAIN and EXPLAIN ANALYZE.
- ✓ Identify performance bottlenecks in real-world queries.
- ✓ Improve performance using indexes, rewrites, and best practices.
- ✓ Measure and compare before-and-after optimization results.
- ✓ Apply enterprise SQL best practices.



Skills You Will Practice

- Writing efficient SQL queries
- Reading and interpreting execution plans
- Using indexes effectively
- Query tuning techniques
- Measuring and validating performance improvements

Lab Workflow

1



Setup and Explore

Load the sample schema and data.
Explore tables, data volumes, and relationships.

2



Run Baseline Queries

Execute baseline queries and capture performance metrics (execution time, rows, buffers).

3



Analyze Execution Plans

Use EXPLAIN ANALYZE to review execution plans.
Identify costly operations and bottlenecks.

4



Optimize Queries

Add or adjust indexes, rewrite queries, and apply best practices to improve performance.

5



Measure and Compare

Rerun queries and compare performance before and after optimization.

6



Document Findings

Record improvements, explain changes, and capture key takeaways.

Environment and Tools



PostgreSQL
16+



psql / SQL Editor



EXPLAIN
ANALYZE



Performance
Metrics

Dataset

You will work with a sample e-commerce dataset.



customers

Customer master data



orders

Order transactions



order_items

Order line items



products

Product catalog



categories

Product categories



payments

Payment transactions



Expected Outcome

By the end of this lab, you will demonstrate improved query performance with measurable results and clear justifications.



Faster Queries



Efficient Index Usage



Lower Resource Consumption



LAB TASKS AND DELIVERABLES

SQL and Query Performance with PostgreSQL — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md; copy to examples/lab38-crm.
2	Ensure Lab 37 DDL available under database/ddl/.
3	Complete generate → baseline EXPLAIN → indexes → optimized queries.
4	Fill database/performance/report.md with experiment ids lab38-001+.
5	Evidence under notes/screenshots/lab-38/.
6	database/performance/01–05 SQL scripts
7	Volume load with documented skew; fixtures preserved
8	Baseline and after-index EXPLAIN (ANALYZE, BUFFERS) evidence
9	Join + CTE + aggregation examples
10	Sargable predicate rewrite (no function on indexed column)
11	OFFSET vs keyset pagination demos
12	N+1 awareness note (app loop vs JOIN/CTE)

EXPECTED DELIVERABLES

- database/performance/01–05 SQL scripts
Volume load with documented skew; fixtures preserved
Baseline and after-index EXPLAIN (ANALYZE, BUFFERS) evidence
Join + CTE + aggregation examples
Sargable predicate rewrite (no function on indexed column)
OFFSET vs keyset pagination demos
N+1 awareness note (app loop vs JOIN/CTE)
report.md with plan shape, buffers, timing

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor Lab 38; complete pre-lab exercises 1→6 first, then the graded lab.



Lab Tasks and Deliverables

Complete the tasks below using PostgreSQL. Document your findings and submit the deliverables.



LAB TASKS

1



Environment Setup

Connect to the lab PostgreSQL database. Explore the schema, tables, and sample data provided.

2



Run Baseline Queries

Execute the provided slow queries. Capture execution time and review results.

3



Analyze Execution Plans

Use EXPLAIN ANALYZE to examine each query plan. Identify bottlenecks such as sequential scans, sorts, or inefficient joins.

4



Optimize Queries

Improve queries by adding appropriate indexes, rewriting SQL, or both. Apply best practices.

5



Measure and Compare

Rerun the optimized queries. Compare performance metrics before and after optimization.

6



Additional Challenges

Complete the advanced tasks: pagination optimization, eliminating N+1 queries, and improving aggregation performance.

7



Document Findings

Record what you did, why it helped, and the impact on performance. Include execution plans and before/after metrics.

8



Cleanup

Drop any test objects you created. Ensure the database is left in a clean state.



DELIVERABLES

• SQL Scripts

All queries executed (baseline and optimized).

• Execution Plan Analysis

Screenshots or text output of EXPLAIN ANALYZE before and after.

• Performance Comparison Table

Include execution time, rows returned, buffers, and key observations.

• Optimization Summary

List of changes made (indexes, query rewrites, configuration) and why they improved performance.

• Key Takeaways

What you learned about SQL performance and tuning.



SUCCESS CRITERIA



All baseline queries executed and analyzed.



At least 3 queries significantly improved ($\geq 50\%$ reduction in execution time).



Appropriate indexes created and justified.



Execution plans show improvement (e.g., index scans instead of sequential scans).



All deliverables complete and clearly documented.



PERFORMANCE METRICS TO CAPTURE



Execution Time
(ms)



Rows Returned



Rows Scanned



Buffers (Shared
Hit/Read)



Sort Method
& Disk Usage



TIPS

- Always test with realistic data volumes.
- Run each query multiple times.
- Use ANALYZE after adding indexes.
- Focus on the biggest bottlenecks first.
- Keep your changes minimal and justified.



Provided Resources



Schema SQL Script
(create_schema.sql)



Sample Data Script
(load_data.sql)



Baseline Queries
(baseline_queries.sql)



Indexes Reference Guide
(index_guide.md)



Best Practices Guide
(sql_best_practices.md)



MODULE 38 SUMMARY

You can now read a PostgreSQL execution plan and fix the query it describes.

KEY CONCEPTS COVERED



Joins & Subqueries

INNER, LEFT, multiple joins, subqueries, correlated subqueries, and CTEs.



Aggregation

GROUP BY, HAVING, and window functions for analytical queries.



Indexing Deep Dive

Selectivity, column order, partial, and expression indexes.



Query Planner

The cost-based optimizer, EXPLAIN, and EXPLAIN ANALYZE.



Scans & Joins

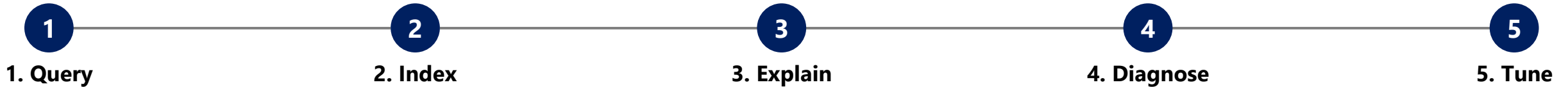
Sequential, index, and bitmap scans; nested loop, hash, and merge joins.



Tuning Workflow

A repeatable process: measure, diagnose, fix, re-measure.

MODULE FLOW RECAP



KEY TAKEAWAY You can now write efficient SQL, choose the right index, read an EXPLAIN ANALYZE plan, and apply a repeatable tuning workflow to a slow PostgreSQL query.

Module Summary

You can write efficient, scalable SQL and optimize query performance in PostgreSQL by understanding how queries are planned and executed.



What We Covered



Query Foundations

SELECT, filtering, joins, subqueries, CTEs, aggregations, and window functions.



Indexing Fundamentals

How indexes work, selectivity, composite, partial, and expression indexes.



Query Planner and Optimizer

PostgreSQL cost-based optimizer, statistics, cardinality, and cost estimation.



Execution Plans

Using EXPLAIN and EXPLAIN ANALYZE to read and interpret execution plans.



Join Strategies

Nested Loop, Hash Join, and Merge Join and when each is used.



Performance Best Practices

Avoiding anti-patterns, writing efficient SQL, and keeping statistics up to date.

Key Takeaways



Great performance starts with good SQL design and understanding your data.



Indexes are powerful—use the right type and column order.



Always examine execution plans to find bottlenecks.



Small changes (indexes, predicates, rewrites) can lead to big improvements.



Measure before and after—let data guide your decisions.



Follow enterprise SQL best practices for consistent, scalable performance.

Common Performance Killers



Functions on indexed columns



N+1 query problem



Over-indexing



SELECT *



Missing or outdated statistics



Inefficient pagination (large OFFSET)

Result



Better Queries
Better Performance
Better Applications

Measure. Analyze. Optimize. Repeat.



Remember

SQL performance is a continuous process of learning, measuring, and improving. Keep analyzing, keep tuning, and keep optimizing!



Understand your data



Measure constantly



Optimize intelligently



Iterate and improve



KNOWLEDGE CHECK (1 OF 2)

Test your understanding of joins, subqueries, and aggregation.

1. Which join returns every row from the left table, with NULLs where no match exists on the right?

- A. INNER JOIN
- B. LEFT JOIN**
- C. A junction table
- D. CROSS JOIN

3. Which clause filters groups after GROUP BY has already aggregated the rows?

- A. WHERE
- B. HAVING**
- C. ORDER BY
- D. LIMIT

2. A subquery that references a column from its outer query is called:

- A. A CTE
- B. A correlated subquery**
- C. A window function
- D. A materialized view

4. A CTE, written with WITH name AS (...), is best described as:

- A. A permanent table
- B. A named, temporary result set scoped to one query**
- C. An index type
- D. A stored procedure

KEY TAKEAWAY LEFT JOIN keeps every left-side row; a correlated subquery references the outer query; HAVING filters after GROUP BY; a CTE is a named, query-scoped result set.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on indexing, execution plans, and query tuning.

1. A composite index on (status, created_at) is most useful for a query filtering on:

- A. created_at alone
- B. status alone, or status and created_at together**
- C. created_at alone, or created_at and status together
- D. Neither column

3. A Seq Scan appearing on a large table for a highly selective WHERE clause most likely means:

- A. The query is already optimal
- B. A useful index is missing or not being used**
- C. The table has too many indexes
- D. PostgreSQL never uses Seq Scan

2. Which PostgreSQL command shows a query's actual execution time and real row counts, not just estimates?

- A. EXPLAIN
- B. EXPLAIN ANALYZE**
- C. ANALYZE VERBOSE
- D. DESCRIBE

4. Offset pagination becomes slow on deep pages mainly because:

- A. PostgreSQL cannot sort large result sets
- B. The database must still count and skip every prior row before returning the page**
- C. OFFSET is not valid SQL syntax
- D. LIMIT stops working past page 10

KEY TAKEAWAY A composite index serves its leading column, alone or combined with later columns; EXPLAIN ANALYZE shows real timings; an unexpected Seq Scan usually means a missing or unused index; deep OFFSET pages are slow because prior rows must still be skipped.



1 Which statement is true about EXPLAIN ANALYZE?

- ☐ A It only shows the estimated plan.
- ☐ B It shows the actual execution plan and runtime statistics.
- ☐ C It modifies the data in the table.
- ☐ D It improves query performance.

2 Which of the following is a benefit of indexes?

- ☐ A Faster data modification
- ☐ B Reduces disk space usage
- ☐ C Improves query performance
- ☐ D Guarantees unique values

3 Which join type returns all rows from the left table and matching rows from the right table?

- ☐ A INNER JOIN
- ☐ B LEFT JOIN
- ☐ C RIGHT JOIN
- ☐ D FULL OUTER JOIN

4 What is the N+1 query problem?

- ☐ A Running a query without an index
- ☐ B Executing N additional queries for related data
- ☐ C Joining more than one table
- ☐ D Not using ANALYZE

5 Which command is used to update statistics in PostgreSQL?

- ☐ A UPDATE STATS;
- ☐ B REINDEX;
- ☐ C ANALYZE;
- ☐ D REFRESH;

6 Which scan type is typically the most efficient for large tables when using an index?

- ☐ A Sequential Scan
- ☐ B Index Scan
- ☐ C Index Only Scan
- ☐ D Bitmap Heap Scan

7 Which of the following is a best practice for better performance?

- ☐ A SELECT * from large tables
- ☐ B Use functions on indexed columns
- ☐ C Use appropriate indexes
- ☐ D Avoid WHERE clause in queries

8 What is keyset pagination preferred over offset pagination?

- ☐ A Easier to implement
- ☐ B Supports random page jumps
- ☐ C More consistent and faster for large datasets
- ☐ D Works with any sorting column



Challenge Question

```
SELECT o.id, o.total_amount, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE DATE(o.created_at) = '2024-05-01'
AND o.status = 'COMPLETE'
ORDER BY o.created_at DESC;
```

Which index would most improve the performance of this query?

- ☐ A idx_orders_status
- ☐ B idx_orders_created_at_status
- ☐ C idx_orders_customer_id
- ☐ D idx_customers_name



True or False

1. Using functions on indexed columns prevents the index from being used.
2. Indexes always improve performance.
3. VACUUM reclaims storage and improves performance.
4. Composite index column order matters.



Answer Hint

Review the module sections on execution plans, indexes, joins, statistics, and best practices to find the correct answers.



Check Your Answers

Compare your answers with the instructor or answer key after attempting the questions.

Key Takeaways from This Module



Understand how queries are executed and optimized.



Use indexes strategically for better performance.



Analyze execution plans to find bottlenecks.



Follow best practices to write efficient and scalable SQL.



Measure, analyze, optimize, and repeat continually.



MODULE 38 COMPLETE

SQL and Query Performance with PostgreSQL

You can now write efficient SQL, index deliberately, and read a PostgreSQL EXPLAIN ANALYZE plan to diagnose and fix a slow query.